



Compiler Design (UE23CS341B)

Chapter 3 – Lexical Analysis

Prakash C O

Department of Computer Science & Engineering

Compiler Design (UE23CS341B)

Chapter 3 – Lexical Analysis

1. The Role of the Lexical Analyzer
2. Input buffering
3. Specification of Tokens.
4. Recognition of Tokens
5. Introduction to LEX tool.
6. Design of a Lexical Analyzer Generator.

Prakash C O

Department of Computer Science & Engineering



The Role of the Lexical Analyzer

- **The main task of the lexical analyzer is to**
 - 1. Read the input characters of the source program, group them into meaningful units called lexemes, and**
 - 2. Produce as output a sequence of tokens for each lexeme in the source program.**
- **The stream of tokens generated by the Lexical Analyzer is sent to the parser for syntax analysis.**
- **When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter that lexeme into the symbol table.**

The Role of the Lexical Analyzer

- The interactions between Lexical analyzer, symbol table and parser are suggested in Fig. 3.1.

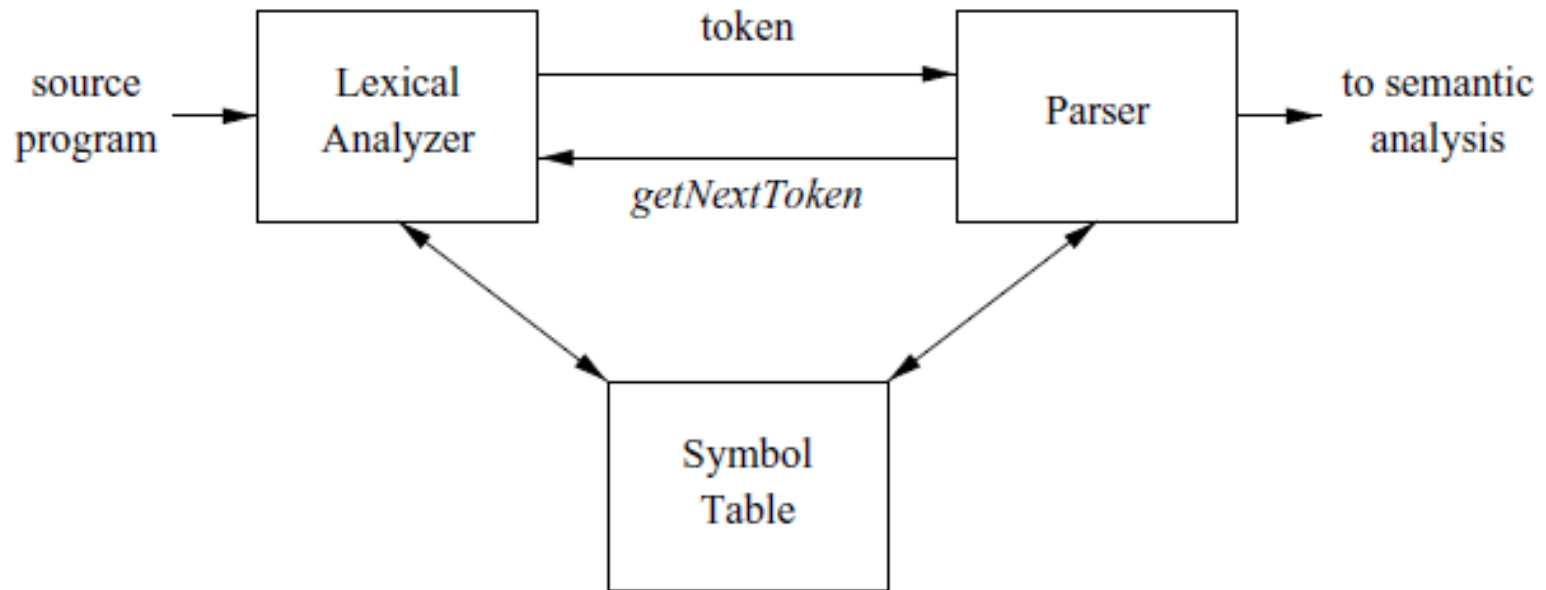


Figure 3 1 Interactions between the lexical analyzer and the parser

The Role of the Lexical Analyzer

- **The lexical analyzer may perform certain other tasks besides identification of lexemes.**
- **One such task is stripping out comments and whitespaces.**
(**blank, newline, tab**, and perhaps other characters that are used to separate tokens in the input).
- **Another task is correlating error messages generated by the compiler with the source program.**

For instance, **the lexical analyzer may keep track of the number of newline characters seen, so it can associate a line number with each error message.**

The Role of the Lexical Analyzer

- **The lexical analyzer may perform certain other tasks besides identification of lexemes.** Cont..
- **In some compilers,**
 - **If the source program uses a macro, the expansion of macros may also be performed by the lexical analyzer.**
 - **the lexical analyzer makes a copy of the source program with the error messages inserted at the appropriate positions.**

The Role of the Lexical Analyzer

➤ Sometimes, lexical analyzers are divided into a cascade of two processes:

a) **Scanning** consists of the simple processes such as

- deletion of comments and
- compaction of consecutive whitespace characters into one.

b) **Lexical analysis** is the more complex portion, where the scanner produces the sequence of tokens as output.

The Role of the Lexical Analyzer

➤ Home work:

1. **Write a Lex program to remove comment lines and also count the number of comments in a program.**

Take as input the lex.yy.c file and eliminate all the comments.

2. **Write a program in a preferred programming language to develop a simple lexical analyzer.**

- ▣ Input : C/C++ program

- ▣ Output : Remove comments ; Generate Tokens; Store generated tokens in table(Symbol table).

The Role of the Lexical Analyzer

Lexical Analysis Versus Parsing

- There are a number of reasons why the analysis portion of a compiler is normally separated into
 1. **Lexical Analysis** and
 2. **Syntax Analysis (parsing) phases.**

The Role of the Lexical Analyzer

Lexical Analysis Versus Parsing

1. Simplicity of design is the most important consideration.

- The separation of lexical and syntactic analysis often allows us to simplify at least one of these tasks.

For example, **a parser that had to deal with comments and whitespace as syntactic units would be considerably more complex** than one that can assume comments and whitespace have already been removed by the lexical analyzer.

- **If we are designing a new language, separating lexical and syntactic concerns can lead to a cleaner overall language design.**

Lexical Analysis Versus Parsing

2. **Compiler efficiency is improved.** A separate lexical analyzer allows us to **apply specialized techniques** that serve only the lexical task, not the job of parsing.

In addition, **specialized buffering techniques for reading input characters can speed up the compiler significantly.**

3. **Compiler portability is enhanced.** Input-device-specific peculiarities can be restricted to the lexical analyzer.

The Role of the Lexical Analyzer

Tokens, Patterns, and Lexemes

1) Token

- A token is a pair consisting of a token name and an optional attribute value.

<token-name, attribute-value>

- Token name is an abstract symbol representing a kind of lexical **unit**. The token names are the input symbols that the parser processes.
- Examples:
 - A particular keyword(E.g. <IF> , <WHILE>), or
 - a sequence of input characters denoting an identifier.

The Role of the Lexical Analyzer

Tokens, Patterns, and Lexemes

- A **pattern (or Regex)** is a description of the form that the lexemes of a token may take.

For **keywords**, the pattern is just the sequence of characters that form the keyword.

if {printf("\nkeyword\n"); return IF;}

while {printf("\nkeyword\n"); return WHILE;}

For **identifiers** and some other tokens, the pattern is a more complex structure that is matched by many strings.

[a-zA-Z_] [a-zA-Z0-9_]* {printf("\nValid input\n"); yylval = yytext; return ID;}

- A **lexeme** is a sequence of characters in the source program that matches the pattern for a token and is identified by the lexical analyzer as an instance of that token.

The Role of the Lexical Analyzer

- Table below gives some **typical tokens**, their informally described patterns, and some sample lexemes.

TOKEN NAME	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters i, f	if
else	characters e, l, s, e	else
comparison	< or > or <= or >= or == or !=	<=, !=
id	letter followed by letters and digits	pi, score, D2
number	any numeric constant	3.14159, 0, 6.02e23
literal	anything but ", surrounded by " 's	"core dumped"

Figure 3.2: Examples of tokens

The Role of the Lexical Analyzer

- **Example 3.1:** Find the number of lexemes/tokens in the below C statement

```
printf("Total = %d\n", score);
```

The Role of the Lexical Analyzer

- **Example 3.1:** Find the number of lexemes/tokens in the below C statement

```
printf("Total = %d\n", score);
```

- both **printf** and **score** are lexemes matching the pattern for token **id**, and
- **"Total = %d\n"** is a lexeme matching the pattern for token **literal**.

Answer: Total number of lexemes/tokens are 7

The Role of the Lexical Analyzer

- **Example 3.2:** Find the number of lexemes/tokens in the below C++ statement

```
cout << "Value of global x is " << ::x;
```

The Role of the Lexical Analyzer

- **Example 3.2:** Find the number of lexemes/tokens in the below C++ statement

```
cout << "Value of global x is " << ::x;
```

Answer: Total number of lexemes/tokens are 7

Note: In C++ program we can access a global variable using scope resolution operator `::` when there is a local variable with same name.

The Role of the Lexical Analyzer

In many programming languages, the **following classes cover most or all of the tokens**:

1. One token for each **keyword**. The pattern for a keyword is the same as the keyword itself.
2. Tokens for the **operators**, either individually or in classes such as the token comparison mentioned in Fig. 3.2.
3. One token representing all **identifiers**.
4. One or more tokens representing constants, such as numbers and **literals**
5. Tokens for each **punctuation symbol**, such as left and right parentheses, left and right square brackets, comma, semicolon ...

Attributes for Tokens

- When **more than one lexeme can match a pattern**, the lexical analyzer **must provide the subsequent compiler phases additional information** about the particular lexeme that matched.
- For example, the **pattern for token number** matches both 10 and 11, but it is extremely important for the code generator to know which lexeme was found in the source program. Tokens: **<num, 10>** , **<num, 11>**, ...

Example: pattern for numbers

The Role of the Lexical Analyzer

Attributes for Tokens

- When more than one lexeme can match a pattern, the lexical analyzer must provide the subsequent compiler phases additional information about the particular lexeme that matched.
- For example, the **pattern for token number** matches both 10 and 11, but it is extremely important for the code generator to know which lexeme was found in the source program. Tokens: **<num, 10>** , **<num, 11>**, ...

Example: pattern for numbers

digit → [0-9]

digits → **digit**⁺

number → **digits** (\. **digits**)? | **digit** \. **digits** E [+ -]? **digits**

Attributes for Tokens

- The lexical analyzer returns to the parser a token;
 - the token name influences parsing decisions, and
 - the attribute value influences translation of tokens after the parse.

The Role of the Lexical Analyzer

Attributes for Tokens

- Assume that tokens have at most one associated attribute, although this attribute may have a structure that combines several pieces of information.
- The most important example is the token **id**, where we need to associate with the token a great deal of information.

Normally, information about an identifier — e.g., its lexeme, its type, and the location at which it is first found is kept in the symbol table.

Thus, **the appropriate attribute value for an identifier is a pointer to the symbol-table entry for that identifier.**

The Role of the Lexical Analyzer

- **Example 3.2** : The token names and associated attribute values for the Fortran statement `E = M * C ** 2` are written below as a sequence of pairs.

E <id, pointer to symbol-table entry for E>

= <assign_op, = > **or** <assign_op> **or** <=>

M <id, pointer to symbol-table entry for M>

***** <arith_op, *> **or** <mult_op> **or** <*>

C <id, pointer to symbol-table entry for C>

****** <exp_op> **or** <*>

2 <number, 2>

- Note that in certain pairs, **especially operators, punctuation, and keywords, there is no need for an attribute value**. In this example, the token *number* has been given an integer-valued attribute.

Lexer implementation is language dependent

Syntactic Sugar:

- Syntactic sugar is syntax within a programming language that is designed to make things easier to read or to express.
- Examples:
 - In C : **b[j]** is a syntactic sugar for ***(b+j)**
 - In C : **x+=y** is equivalent to **x = x + y**
 - In C# : **var x = expr** (compiler deduces type of x from expr)
- Compilers expand sugared constructs into fundamental constructs (Desugaring).

Lexical Analyzer

Scanning/Lexical Analysis is Hard in C++

because the same characters (>, <, >>, <<) are overloaded with multiple meanings depending on context.

■ **Ambiguity of tokens:** The same symbol can represent

■ **Relational operators:** <, >

■ **Bitwise shifts:** <<, >>

■ **Stream operators:** cin >> var;

■ **Template delimiters:** template <class T>

■ **Nested templates:** A<B<C>>D;

In the above nested template >> is not a right-shift operator instead it is two closing template brackets.

How C++ Compilers Handle It

- Unlike many languages where token meaning is fixed, **in C++ the meaning of < or > depends on syntactic context.**

For example:

- `a < b` → comparison
- `vector <int>v;` → template argument closing
- `a >> 4` → bitwise shift

- In C++, the lexer often needs **parser feedback** to decide whether `>>` is a shift operator or two closing template brackets.

`A<B<C>>D;`

Scanning/Lexical Analysis is Hard in PL/I

■ In PL/I, keywords are not reserved

In PL/I, keywords are **context-sensitive**: the same word can be a keyword in one place and a programmer-defined identifier in another.

■ Context determines meaning

The lexer cannot simply mark IF as a keyword every time it appears. Instead, the parser must look at the surrounding tokens:

- If IF appears at the start of a statement → it's the control keyword.
- If IF appears in a declaration → it could be a variable name.

■ Example:

```
DECLARE IF FIXED BIN;
```

```
IF IF > 0 THEN PUT SKIP LIST('Positive');
```

Lexical Analyzer

■ Scanning/Lexical Analysis is Hard in Python

Consider a Python block of code.

```
if w==z:
    a=b
    c=d
else:
    e=f
g=h
```

In python, block structure is handled by whitespaces (i.e., indentation).

In Python,

- Indentation is not ignored.
- Indentation defines block structure.
- Leading whitespace is used to compute a line's indentation level, which in turn is used to determine the block of statements (i.e., code block).

Lexical Analyzer

■ Scanning/Lexical Analysis is Hard in Python

- In Python lexical analysis, most of the white space magic is in the lexer, **the lexer emits three special tokens: NEWLINE, INDENT, and DEDENT.**
- The parser reads
 - **NEWLINE** as the end of a logical line,
 - **INDENT** as the beginning of a block and
 - **DEDENT** as the end of a block.
- Note that **INDENT and DEDENT encode change in indentation, not the total amount of indentation.**

Lexical Analyzer

■ Scanning/Lexical Analysis is Hard in Python

```
if w==z:  
    a=b  
    c=d  
else:  
    e=f  
g=h
```

Lexical Analyzer

Scanning Python

```
if w==z:
```

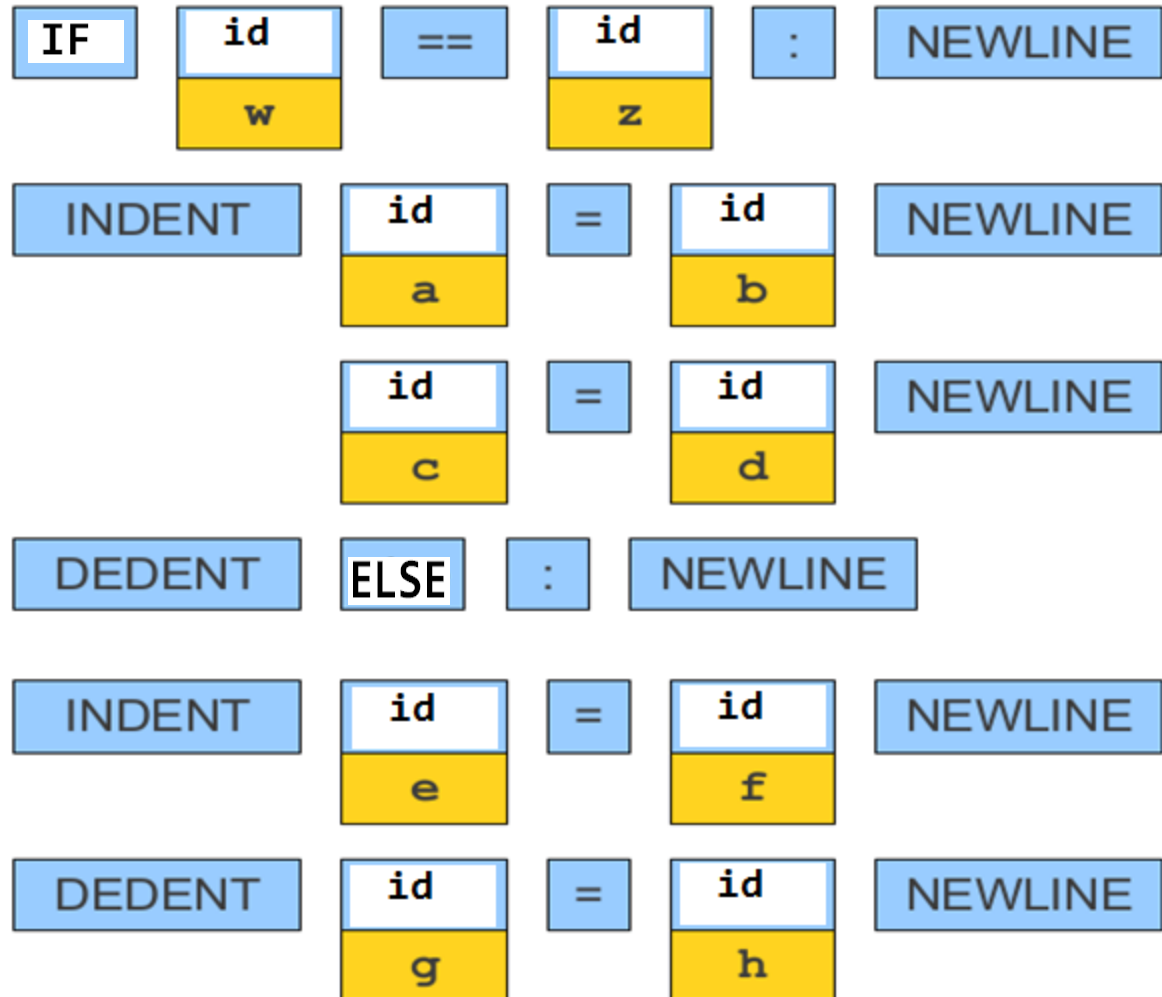
```
    a=b
```

```
    c=d
```

```
else:
```

```
    e=f
```

```
g=h
```



Lexical Analyzer

■ How INDENT and DEDENT tokens are generated in Python Lexical Analyzer?

- Before the first line of the file is read, a single zero is pushed on to stack;
this will never be popped off again. The numbers pushed on the stack will always be strictly increasing from bottom to top.
- At the beginning of each logical line, the line's indentation level is compared to the stack top value.
 - If it is equal, nothing happens.
 - If it is larger, it is pushed on to the stack, and one INDENT token is generated.
 - If it is smaller, it *must* be one of the numbers occurring on the stack; all numbers on the stack that are larger are popped off, and for each number popped off a DEDENT token is generated.
- At the end of the file, a DEDENT token is generated for each number remaining on the stack that is larger than zero.

Lexical Analyzer

- Example of a correctly (though confusingly) indented piece of Python code:

```
def perm(l):  
    # Compute the list of all permutations of l  
    if len(l) <= 1:  
        return [l]  
  
    r = []  
    for i in range(len(l)):  
        s = l[:i] + l[i+1:]  
        p = perm(s)  
        for x in p:  
            r.append(l[i:i+1] + x)  
    return r
```

https://docs.python.org/3/reference/lexical_analysis.html

Lexical Analyzer

- **How INDENT and DEDENT tokens are generated in Python Lexical Analyzer?**
 - Before the first line of the file is read, a single zero is pushed on the stack; At the beginning of each logical line, the line's indentation level is compared to the top of the stack.
 - If it is equal, nothing happens.
 - If it is larger, it is pushed on to the stack, and one INDENT token is generated.
 - If it is smaller, it *must* be one of the numbers occurring on the stack; all numbers on the stack that are larger are popped off, and for each number popped off a DEDENT token is generated.
 - At the end of the file, a DEDENT token is generated for each number remaining on the stack that is larger than zero.

```
def perm(l):  
    # Compute the list of all permutations of l  
    if len(l) <= 1:  
        return [l]  
    r = []  
    for i in range(len(l)):  
        s = l[:i] + l[i+1:]  
        p = perm(s)  
        for x in p:  
            r.append(l[i:i+1] + x)  
    return r
```


Lexer Feedback

- C and C++ lexers require lexical feedback to differentiate between *typedef names* and *identifiers*.

That is, the context sensitive lexer needs help from the “context-free” parser to distinguish between an identifier “foo” and a typedef name “foo”. In this snippet,

```
int foo;           // here “foo” is an identifier
typedef int foo;   // here “foo” is an typedef name
foo x;            // here “foo” is an typedef name
```

the first “foo” is an identifier while the second and third are typedef names. **You need to parse typedef declarations to figure this out.**

- This means that the parser and lexer are mutually recursive, so it doesn’t make sense to say that the parser is context free while the lexer is context sensitive.

Lexical Analyzer

■ How C/C++ parser/lexer makes the difference between '*' of pointer and '*' of multiplication?

```
... {  
    typedef int obj;  
    obj *var1;      // * used to make var1 as pointer to obj  
    var3 = var1 * var2; // * used to multiply var1 and var2  
}
```

- The **tokenizer (or Lexer)** doesn't make a distinction between the two. It just treats it as the token `<*>`.
- **The parser knows how to look up names.** It knows that `obj` is a type, so can parse `<type> * <identifier>` differently from `<non-type> * <non-type>`.
- When `typedef int obj;` is parsed, it is interpreted and taken to mean `obj` now names a type. When parsing continues and `obj *var1;` is seen, the results of the earlier semantic analysis are available for use.

▣ Contextual keywords

- ▣ Sometimes, we want to add a new keyword to a language, but can't break backward compatibility.

For example, when C# added *async/await*, they needed to retain *async* as an identifier, so that old programs that used variables or classes named *async* would still compile.

That is to say, the token *async* should be treated as an identifier under some circumstances, and as a keyword under others.

A keyword with this property is called a “**contextual keyword**” in C#.

- ▣ Then the question is: when should we lex the string “*async*” appearing in the source code as an identifier, and when should we lex it as the *async* token?

We need information about the surrounding tokens to decide that.

The Ambiguity in Fortran Lexical Analysis

▣ Statement 1: **DO 5 I = 1.25**

- Because **blanks are ignored** in fixed-format Fortran, this line is equivalent to: `DO5I=1.25`
- Here, the first lexeme is **DO5I**, which is treated as an **identifier** (a variable name).
- The statement is an **assignment**: Assign the floating-point value 1.25 to the variable DO5I.

Note: keywords are *not* reserved in Fortran. In Fortran, words like DO, IF, END, REAL, etc. are recognized as keywords only by context.

The Ambiguity in Fortran Lexical Analysis

■ Statement 2: **DO 5 I = 1,25**

```
do label var = expr1, expr2, expr3  
statements  
label continue
```

- Again, blanks are ignored, so this is equivalent to: `D05I=1,25`
- But the presence of the **comma** changes everything.
- Now the lexer recognizes the keyword **DO**, followed by the label 5, the loop variable I, and the range 1 to 25.
- This is a **DO loop**:
Execute the block labeled 5 with I running from 1 to 25.

The Ambiguity in Fortran Lexical Analysis

■ Why Lookahead Is Needed

- Until the lexer sees the **dot** in 1.25, it cannot be sure whether the statement is:
 - An **assignment** (`D05I = 1.25`)
 - Or a **loop** (`DO 5 I = 1,25`)
- This requires **multi-character lookahead**: the lexer must scan ahead to disambiguate tokens.
- In modern compiler design, this is considered undesirable because it complicates the lexical phase.

Tricky Problems When Recognizing Tokens

- Lexical analysis is complicated in some languages. FORTRAN, for example, allows white space inside of lexemes. The FORTRAN statement:

DO 5 I = 1.25

- is an assignment statement with three tokens:

ID	ASSIGNOP	NUM
DO5I	=	1.25

- but the FORTRAN statement:

DO 5 I = 1,25

- is a DO-statement with seven tokens:

DOTOK	NUM	ID	ASSIGNOP	NUM	COMMA	NUM
DO	5	I	=	1	,	25

- Before the lexical analyzer can produce the first token it must look ahead to see if there is a dot or a comma in this statement.

The Role of the Lexical Analyzer

Lexical Errors

- It is hard for a lexical analyzer to tell, without the aid of other components, that there is a source-code error.
- For instance, if the string **fi** is encountered for the first time in a C program in the context:

```
fi(a == f(x))  
printf(a)
```

a lexical analyzer cannot tell whether **fi** is a misspelling of the keyword **if** or an undeclared function identifier.

Since **fi** is a valid lexeme for the token **id**, the lexical analyzer must return the token **id** to the parser and let some other phase of the compiler — probably the parser in this case — handle an error due to transposition of the letters.

Lexical Errors

- A **lexical error** is any input that can be rejected by the lexer. This generally results from token recognition falling off the end of the rules you've defined.
- For example:
 - $[0-9]^+ \implies$ NUMBER token
 - $[a-zA-Z]^+ \implies$ LETTERS token
 - anything else $\implies$ **error**
- If **lexer is a finite state machine that accepts valid input strings**, then errors are going to be any input strings that do not result in that finite state machine reaching an accepting state.

Lexical Errors

▣ Examples:

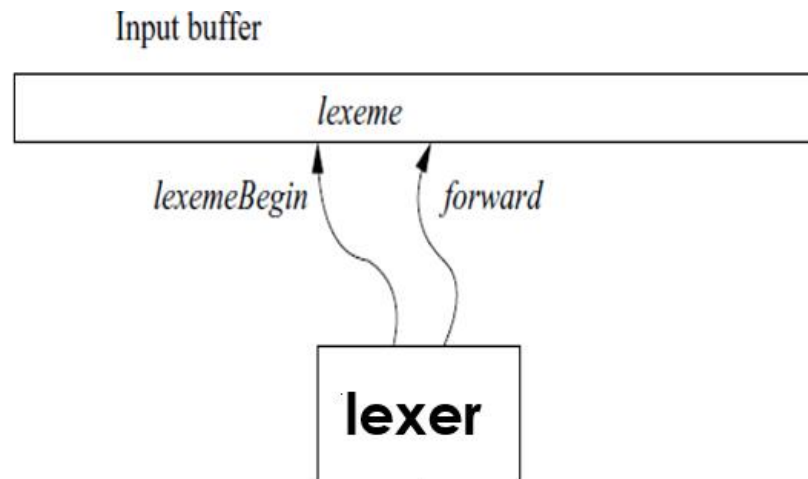
- ▣ A lexical error could be an invalid or unacceptable character by the language, like '@' which is rejected as a lexical error for identifiers in Java (it's reserved).
- ▣ Lexical errors are the errors thrown by your lexer when unable to continue. Which means that there's no way to recognize a *lexeme* as a valid *token* for your lexer.

Input Buffering

Input Buffering is a technique used to efficiently read characters from the source program into memory for lexical analysis.

Input Buffering

- Let us examine some ways that the simple but important task of reading the source program (from disk to memory) can be speeded.
- This reading task is made difficult by the fact that we often have to look one or more characters beyond the next lexeme before we can be sure we have the right lexeme.



Input Buffering

- **There are many situations where we need to look at least one additional character ahead in determining a right lexeme.**

Input Buffering

- **There are many situations where we need to look at least one additional character ahead in determining a right lexeme.**
 1. For instance, we cannot be sure we've seen the **end of an identifier until we see a character that is not a letter or digit**, and therefore is not part of the lexeme for **id**.
 2. In C, single-character operators like **-, =, or <** could also be the beginning of a two-character operator like **->, ==, or <=**.
Thus, a **two-buffer scheme** that **handles large lookaheads safely** is introduced.

We then consider an improvement involving "**sentinels**" that **saves time checking for the ends of buffers**.

Input Buffering

■ C language Punctuation symbols (delimiters)

- For single-character punctuation symbols like `;`, `,`, `(`, `)`, `{`, `}`, `[`, `]`, **no lookahead is needed** because they are unambiguous and always represent a single token.

■ Two-character operators in C

- `==` (equal to)
- `!=` (not equal)
- `<=`, `>=` (relational)
- `++`, `--` (increment/decrement)
- `&&`, `||` (logical AND/OR)
- `<<`, `>>` (bitwise shift)
- `+=`, `-=`, `*=`, `/=`, etc. (compound assignment)

Input Buffering

- **For C language operators, lookahead is required** because the lexer must check the 2nd character to decide whether:
 - = is a single-character operator (=) or part of two-character operator (==)
 - + is a single-character operator (+) or part of two-character operator (++ or +=)
 - < is a single-character operator (<) or part of two-character operator (<= or <<)

- **Without lookahead, the lexer cannot distinguish between single-character and multi-character operators.**

Input Buffering

- **When the lexer reads the first + in C program, it cannot yet decide:**
 - Is it + (addition)?
 - Or ++ (increment)?
 - Or += (compound assignment)?
- **So it must look ahead one character:**
 - If the next character is +, then the token is ++ (increment).
 - If the next character is =, then the token is +=.
 - Otherwise, the token is +.
- **Once the lexer sees the second +, the decision is final:** ++ is a complete token. There is **no need to look ahead further** because ++ cannot be extended into a longer operator in C.
- **Maximum lookahead for C operators is 1 character.** After reading ++, the lexer immediately returns the increment operator token.

Input Buffering

Buffer Pairs

- The amount of time taken to process characters is more and because of the large number of characters that must be processed during the compilation of a large source program,
specialized buffering techniques have been developed **to reduce the amount of overhead** required to process a single input character.
- An important scheme involves **two buffers** that are **alternately reloaded**, as suggested in Fig. 3.3.

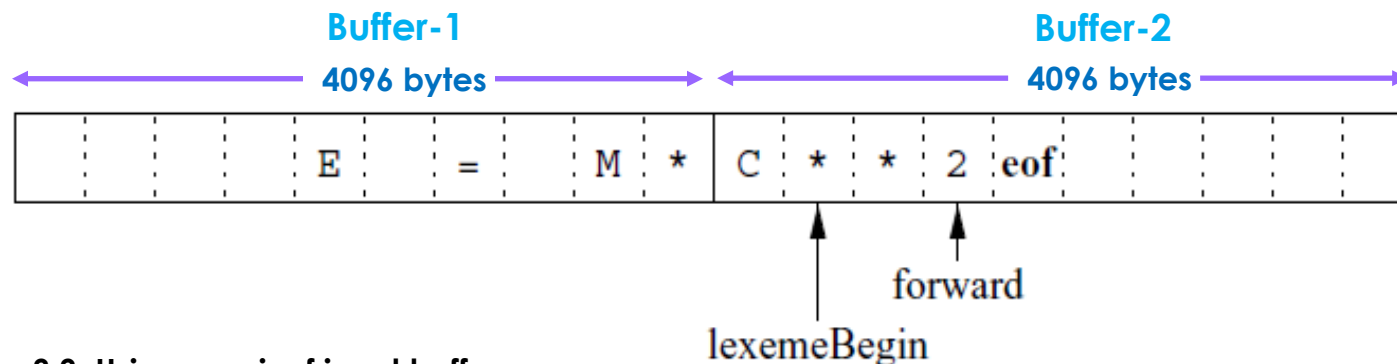


Fig. 3.3. Using a pair of input buffers

Input Buffering

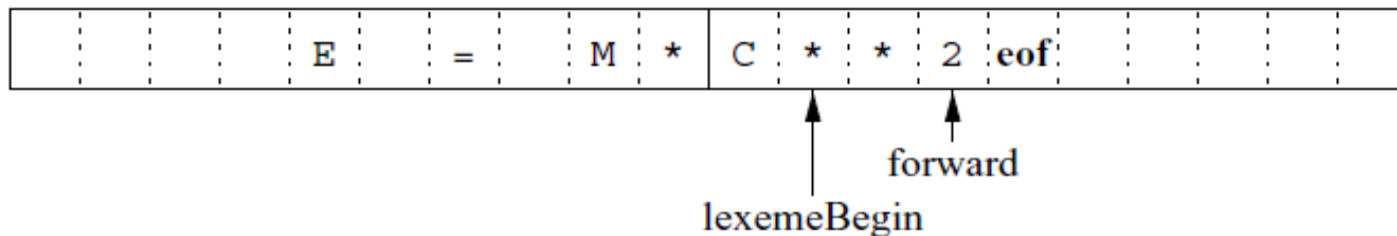
Buffer Pairs

- Each buffer is of the same size N , and N is usually the size of a disk block, e.g., 4096 bytes.
- Using one system read command, we can read N characters into a buffer, rather than using one system call per character.
- If fewer than N characters remain in the input file, then a **special character**, represented by **eof**, marks the end of the source file and is different from any possible character of the source program.

Input Buffering

Buffer Pairs

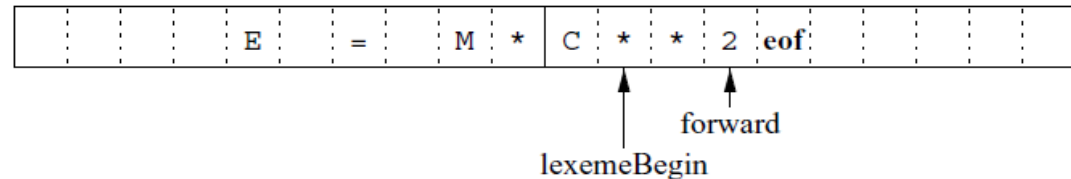
- Two pointers to the input are maintained:
 1. Pointer **lexemeBegin**, marks the beginning of the current **lexeme**, whose extent we are attempting to determine.
 2. Pointer **forward** scans ahead until a pattern match is found;



Input Buffering

Buffer Pairs

- **Once the next lexeme is determined,**
 - In below figure, we see forward has passed the end of the next lexeme, ** (the Fortran exponentiation operator), and **must be retracted one position to its left.**



- **After the lexeme is recorded** as an attribute value of a token returned to the parser,
 - **forward** is set to the character at its right end.
 - **lexemeBegin** is set to the character immediately after the lexeme just found and recorded.

Input Buffering

Buffer Pairs

- **Advancing forward pointer** requires that
 1. we first test whether we have reached the end of one of the buffers, and
 2. if so, we must reload the other buffer from the input, and
 3. move forward to the beginning of the newly loaded buffer.

Input Buffering

Problem with Buffer Pairs:

- When using two buffers (double buffering), for each character read, the lexer normally performs **two checks**:
 - **Check 1:** Is this the end of the buffer?
 - **Check 2:** What is the current character?
- **Optimization:**
 - We can **combine these two checks** by adding a **sentinel character** at the end of each buffer.
 - The **sentinel** is a **special character that cannot appear in the source program**.
 - A natural choice for the sentinel is **EOF (end-of-file)**.

Input Buffering

Sentinels

- Figure 3.4 shows the same arrangement as Fig. 3.3, but with the sentinels added. Note that **eof** retains its use as a marker for the end of the entire input.
- Any **eof** that appears other than at the end of a buffer means that the input is at an end.

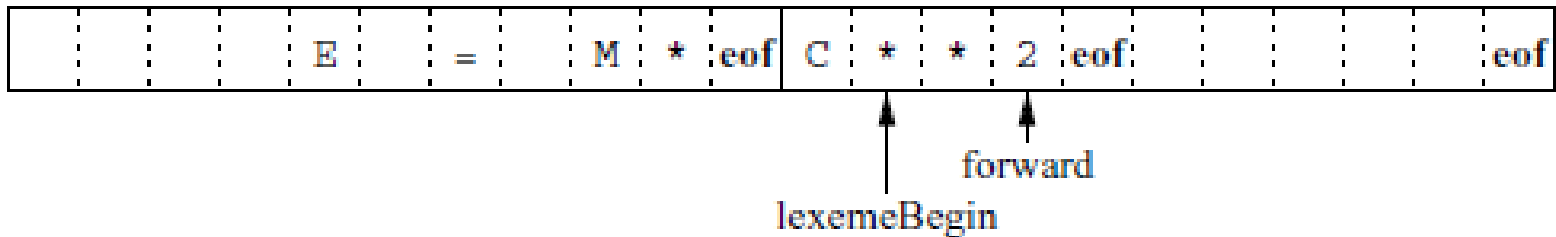


Figure 3.4: Sentinels at the end of each buffer

Sentinels

- Figure 3.5 summarizes the **algorithm for advancing forward**.
 - Notice how the first test, which can be part of a multiway branch based on the character pointed to by forward, is the only test we make, except in the case where we actually are at the end of a buffer or the end of the input.

Input Buffering

Sentinels

```
switch ( *forward++ ) {  
    case eof:  
        if (forward is at end of first buffer ) {  
            reload second buffer;  
            forward = beginning of second buffer;  
        }  
        else if (forward is at end of second buffer ) {  
            reload first buffer;  
            forward = beginning of first buffer;  
        }  
        else /* eof within a buffer marks the end of input */  
            terminate lexical analysis;  
        break;  
    Cases for the other characters  
}
```

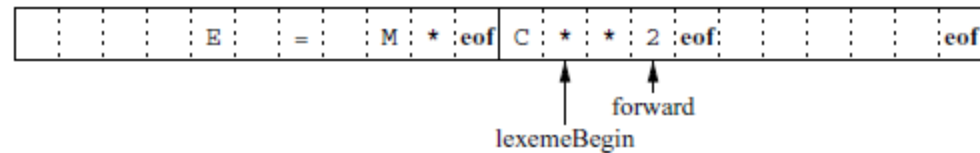


Figure 3.5: Lookahead code with sentinels

Input Buffering

Advantages of Buffer Pairs

- ▣ Reduces I/O overhead.
- ▣ Supports efficient token recognition with lookahead.
- ▣ Provides enough lookahead without storing the entire program (i.e., allows efficient tokenization with minimal memory usage).

Specification of Tokens

Specification of Tokens

- **Regular expressions** are an **important notation** for **specifying lexeme patterns**.
- We shall study the formal notation for regular expressions, and we shall see how these expressions are used in a lexical-analyzer generator.

Specification of Tokens

Regular Expressions

- A **regular expression** is a sequence of characters that define a search/match pattern.
- If **letter_** is established to stand for any letter or the underscore, and **digit** is established to stand for any digit, then we could describe the language of C identifiers by:

letter_ (letter_ | digit)*

Specification of Tokens

Regular Expressions

- The **regular expressions are built recursively out of smaller regular expressions**. Each regular expression r denotes a language $L(r)$, which is also defined recursively from the languages denoted by r 's subexpressions.
- Here are the rules that define the regular expressions over some **alphabet Σ** and the languages that those expressions denote.
- **BASIS: There are two rules that form the basis:**
 1. ϵ is a regular expression, and $L(\epsilon)$ is $\{\epsilon\}$, that is, the language whose sole member is the empty string.
 2. If a is a symbol in Σ , then a is a regular expression, and $L(a) = \{a\}$, that is, the language with one string, of length one, with a in its one position.

Specification of Tokens

Regular Expressions

- **INDUCTION:** There are four parts to the induction whereby larger regular expressions are built from smaller ones.

Suppose **r** and **s** are regular expressions denoting languages $L(r)$ and $L(s)$, respectively.

1. **(r)|(s)** is a regular expression denoting the language $L(r) \cup L(s)$.
2. **(r)(s)** is a regular expression denoting the language $L(r)L(s)$.
3. **(r)*** is a regular expression denoting $L(r)^*$.
4. **(r)** is a regular expression denoting $L(r)$.

This last rule says that we can add additional pairs of parentheses around expressions without changing the language they denote.

Specification of Tokens

Regular Expressions operators precedence

- ▣ The following table gives the order of RE operators precedence, from highest precedence to lowest precedence.

Highest	[]	Character class
	()	Grouping
	* + ? {m, n}	Repetition Operators
	. or No-character operator	Concatenation
	^ \$	Position Anchors
Lowest	 	Alternation (or Union)

- Under these conventions, for example, we may replace the regular expression **(a)|((b)*(c))** by **a|b*c**

Both expressions denote the set of strings that are either a single a or are zero or more b's followed by one c.

Specification of Tokens

Regular Expressions

- Figure 3.7 shows some of the **algebraic laws** that hold for arbitrary regular expressions r , s , and t .

LAW	DESCRIPTION
$r s = s r$	$ $ is commutative
$r (s t) = (r s) t$	$ $ is associative
$r(st) = (rs)t$	Concatenation is associative
$r(s t) = rs rt$; $(s t)r = sr tr$	Concatenation distributes over $ $
$\epsilon r = r\epsilon = r$	ϵ is the identity for concatenation
$r^* = (r \epsilon)^*$	ϵ is guaranteed in a closure
$r^{**} = r^*$	$*$ is idempotent

Regular Definitions

- For notational convenience, we may wish to **give names to certain regular expressions and use those names in subsequent regular expressions, as if the names were themselves symbols.**
- If Σ is an alphabet of basic symbols, then a *regular definition* is a sequence of definitions of the form:

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

.....

$$d_n \rightarrow r_n$$

where:

- Each d_i is a new symbol, not in Σ and not the same as any other of the d 's and
- Each r_i is a regular expression over the alphabet $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$

Specification of Tokens

Regular Definitions

- **Example 3.5:** A regular definition for the language of C identifiers.

Specification of Tokens

Regular Definitions

- **Example 3.5:** A regular definition for the language of C identifiers.

letter_ $\rightarrow$ **A | B | ... | Z | a | b | ... | z | _**

digit $\rightarrow$ **0 | 1 | ... | 9**

id $\rightarrow$ **letter_ (letter_ | digit)***

Specification of Tokens

Regular Definitions

➤ **Example 3.6: Unsigned numbers (integer or floating point)**

are strings such as 5280, 0.01234, 6.336E4, or 1.89E-4.

The regular definitions:

Specification of Tokens

Regular Definitions

➤ Example 3.6: Unsigned numbers (integer or floating point)

are strings such as 5280, 0.01234, 6.336E4, or 1.89E-4.

The regular definitions:

digit $\rightarrow$ 0 | 1 | ... | 9

digits $\rightarrow$ digit digit*

optionalFraction $\rightarrow$ \. digits | ϵ

optionalExponent $\rightarrow$ (E (+ | - | ϵ) digits) | ϵ

number $\rightarrow$ digits optionalFraction optionalExponent

Specification of Tokens

Extensions of Regular Definitions

- Many extensions have been added to regular expressions to enhance their ability to specify string patterns.

1. ***One or more instances.*** *The unary, postfix operator **+** represents the positive closure of a regular expression and its language.*

If r is a regular expression, then $(r)^+$ denotes the language $(L(r))^+$.

The operator $+$ has the same precedence and associativity as the operator $*$.

Two useful algebraic laws, $r^* = r^+ | \epsilon$ and $r^+ = r r^* = r^* r$ relate the **Kleene closure** and **positive closure**.

Specification of Tokens

Extensions of Regular Definitions

2. **Zero or one instance.** The unary postfix operator **?** means "**zero or one occurrence.**" That is, **$r?$ is equivalent to $r|\epsilon$** , or put another way,
- $$L(r?) = L(r) \cup \{\epsilon\}.$$

The **?** operator has the same precedence and associativity as ***** and **+**.

3. **Character classes.** A regular expression **$a_1|a_2|\dots|a_n$** , where the a_i 's are each symbols of the alphabet, can be replaced by the shorthand **$[a_1a_2 \dots a_n]$** .

When **$a_1, a_2, \dots, a_n$** form a logical sequence, we can replace them by **a_1 - a_n** . Thus, **$[abc]$** is shorthand for **$a|b|c$** , and **$[a-z]$** is shorthand for **$a|b| \dots |z$** .

Specification of Tokens

Extensions of Regular Definitions

- Example 3.7 Using these shorthands, we can rewrite the regular definition of Example 3.5 as:

letter_ $\rightarrow$ [A-Za-z_]

digit $\rightarrow$ [0-9]

id $\rightarrow$ letter_ (letter_ | digit)*

- The regular definition of Example 3.6 can also be simplified:

digit $\rightarrow$ [0-9]

digits $\rightarrow$ digit⁺

number $\rightarrow$ digits (\. digits)? | digit \. digits E [+ -]? digits

Recognition of Tokens

Recognition of Tokens

- Now, we must study
 - how to take the patterns for all the needed tokens and
 - build a piece of code that examines the input string and finds a prefix that is a lexeme matching one of the patterns.
- Our discussion will make use of the following running example
 - stmt $\rightarrow$ **if** expr **then** stmt | **if** expr **then** stmt **else** stmt | ϵ
 - expr $\rightarrow$ term **relop** term | term
 - term $\rightarrow$ **id** | **number**

Figure 3.10: A grammar for branching statements in Pascal language

Recognition of Tokens

$\text{stmt} \rightarrow \text{if expr then stmt} \mid \text{if expr then stmt else stmt} \mid \epsilon$

$\text{expr} \rightarrow \text{term relop term} \mid \text{term}$

$\text{term} \rightarrow \text{id} \mid \text{number}$

Figure 3.10: A grammar for branching statements

- **Example 3.8 :** The grammar fragment of Fig. 3.10 describes a simple form of branching statements and conditional expressions.
- **The terminals of the grammar**, which are **if, then, else, relop, id,** and **number**, are the token names in our tokens.

The patterns for these token names are described using regular definitions, as in Fig. 3.11.

Recognition of Tokens

- **Figure 3.11:** Patterns for tokens of Example 3.8

digit → [0-9]

digits → digit⁺

letter_ → [A-Za-z_]

number → digits (\. digits)? | digit \. digits E [+ -]? digits

id → letter_ (letter_ | digit)*

if → if

then → then

else → else

relop → < | > | <= | >= | = | <>

➤ **Note:** In Pascal,
= is “equals” and
<> is “not equals”.

- For this language, the lexical analyzer will recognize the keywords **if**, **then**, and **else** , as well as lexemes that match the patterns for **relop**, **id**, and **number**.

Recognition of Tokens

- The lexical analyzer has an additional job of stripping out whitespace, by recognizing the "token" **ws** defined by:

$$\text{ws} \rightarrow (\text{blank} \mid \text{tab} \mid \text{newline})^+$$

- Here, **blank, tab, and newline are abstract symbols** that we use to express the ASCII characters of the same names.
- **Token ws is different from the other tokens** in that, **when we recognize it, we do not return it to the parser, but rather restart the lexical analysis from the character that follows the whitespace.** It is the following token that gets returned to the parser.

Recognition of Tokens

- Our goal for the lexical analyzer is summarized in Fig. 3.12.

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any ws	-	-
if	if	-
then	then	-
else	else	-
Any id	id	Pointer to table entry
Any number	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
>	relop	GT
>=	relop	GE

Figure 3.12: Tokens, their patterns, and attribute values

- This table shows, for each lexeme or family of lexemes, which token name is returned to the parser and what attribute value is returned.
- Note that for the six relational operators, **symbolic constants** LT, LE, and so on are used as the attribute value, in order to indicate which instance of the token **relop** we have found.

Recognition of Tokens

Transition Diagrams

- As an intermediate step in the construction of a lexical analyzer, **we first convert patterns into stylized flowcharts, called "transition diagrams."**
- Transition diagrams have a collection of nodes or circles, called **states**.

Each state represents a condition that could occur during the process of scanning the input looking for a lexeme that matches one of several patterns.

Recognition of Tokens

Transition Diagrams

- **Edges** are directed from one state of the transition diagram to another.

Each edge is labeled by a symbol or set of symbols.

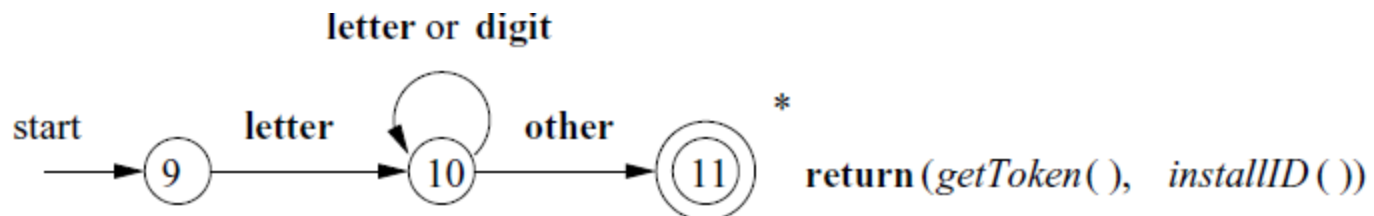
- We shall assume that all our **transition diagrams are deterministic.**

Recognition of Tokens

Transition Diagrams

- Some important conventions about transition diagrams are:
 1. **Accepting, or final states indicate that a lexeme has been found**, although the actual lexeme may not consist of all positions between the lexemeBegin and forward pointers.

In accepting state, if there is an action to be taken — typically returning a token and an attribute value to the parser — we shall attach that action to the accepting state.

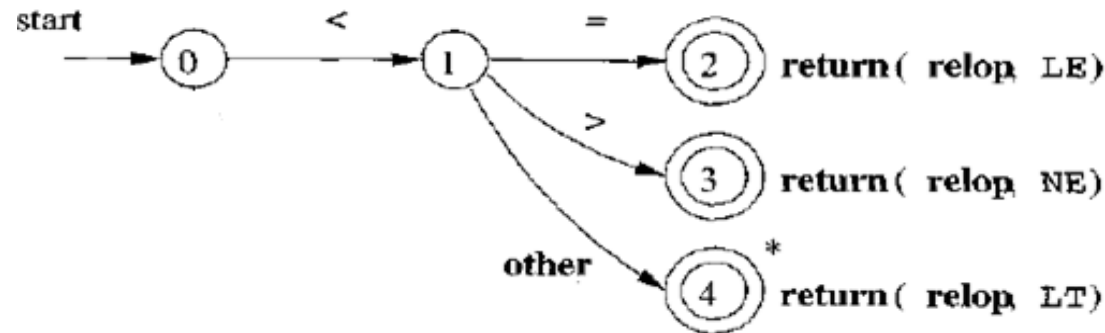


Recognition of Tokens

Transition Diagrams

if (a < b) then

↑ ↑
lexemebegin forward



- Some important conventions about transition diagrams are:
 2. In addition, **if it is necessary to retract the *forward pointer* one position** (i.e., the lexeme does not include the symbol that got us to the accepting state), **then we shall additionally place a *** near that accepting state.

In our example, it is never necessary to retract forward by more than one position, but if it were, we could attach any number of '*'s to the accepting state.

3. One state is designated the **start state**, or **initial state**; it is indicated by an edge, labeled "start," entering from nowhere. The transition diagram always begins in the start state before any input symbols have been read.

Recognition of Tokens

Transition Diagrams

- **Example 3.9 :** Figure 3.13 is a transition diagram that recognizes the lexemes matching the token **relop**.

Recognition of Tokens

Transition Diagrams

- **Example 3.9 :** Figure 3.13 is a transition diagram that recognizes the lexemes matching the token **relop**.

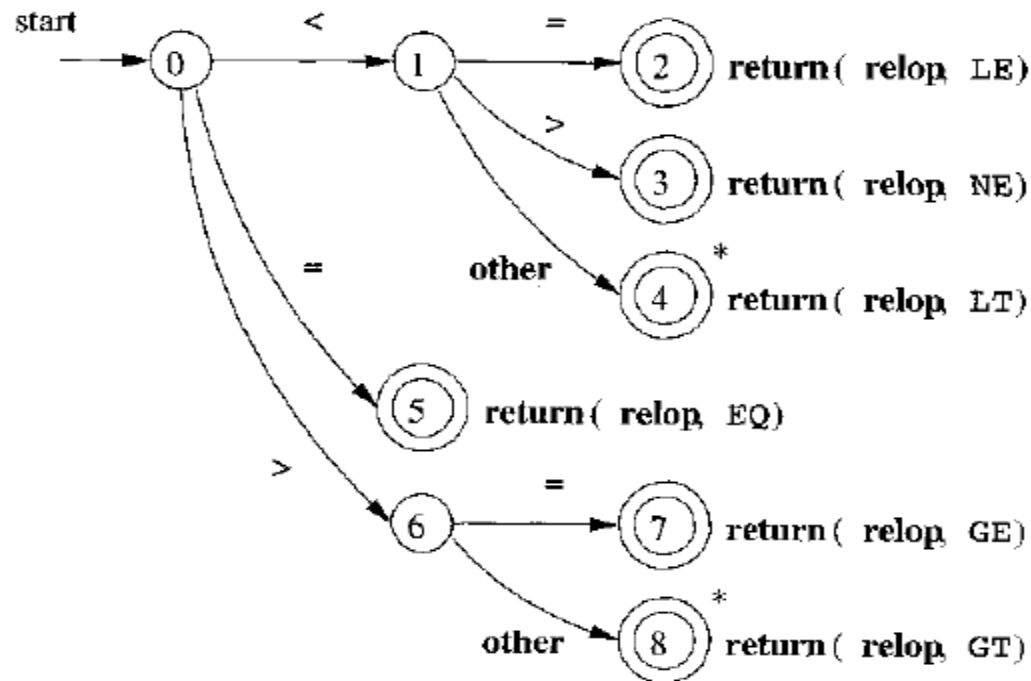


Figure 3.13: Transition diagram for **relop**

Recognition of Tokens

Recognition of Reserved Words and Identifiers

- **Recognizing keywords and identifiers presents a problem.**

Usually, **keywords** like **if** or **then** are reserved, so they **are not identifiers even though they look like identifiers**.

- A transition diagram like that of Fig. 3.14 is used to search for identifier lexemes, will also recognize the keywords **if**, **then**, and **else** of our running example.

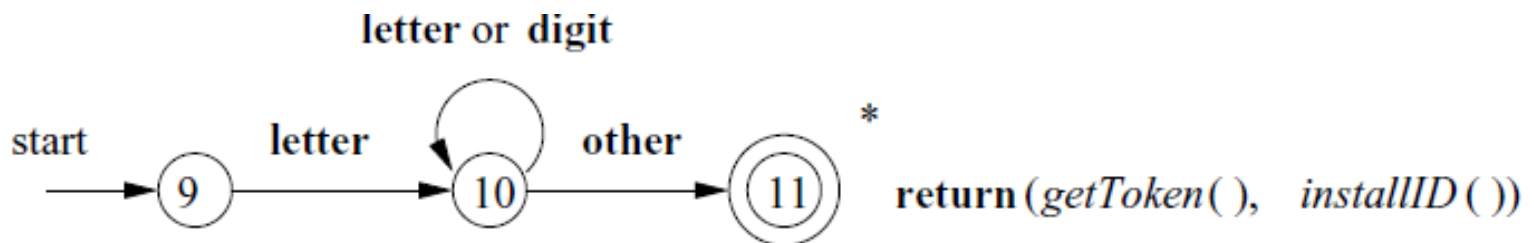


Figure 3.14: A transition diagram for **identifiers** and **keywords**

Recognition of Tokens

Recognition of Reserved Words and Identifiers

➤ There are two ways that we can handle reserved words that look like identifiers:

1. Install the reserved words in the symbol table initially.

A field of the symbol-table entry indicates that

- these strings are never ordinary identifiers, and tells which token they represent.

	lexeme name	token name
1	if	IF
2	then	THEN
3	else	ELSE
4	...	...
53	sum	ID
54	avg	ID

This method is in use in Fig. 3.14. When we find an identifier, a call to *installID*

- places it in the symbol table if it is not already there and returns a pointer to the symbol-table entry for the lexeme found.

Any identifier not in the symbol table during lexical analysis cannot be a reserved word, so its token is id (or ID).

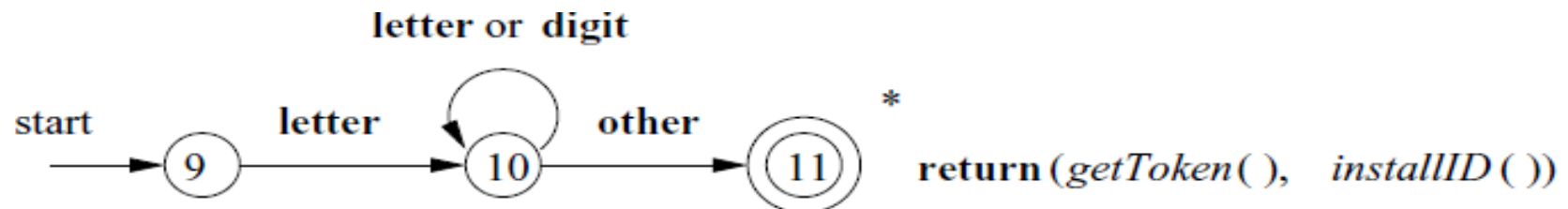


Fig. 3.14

Recognition of Tokens

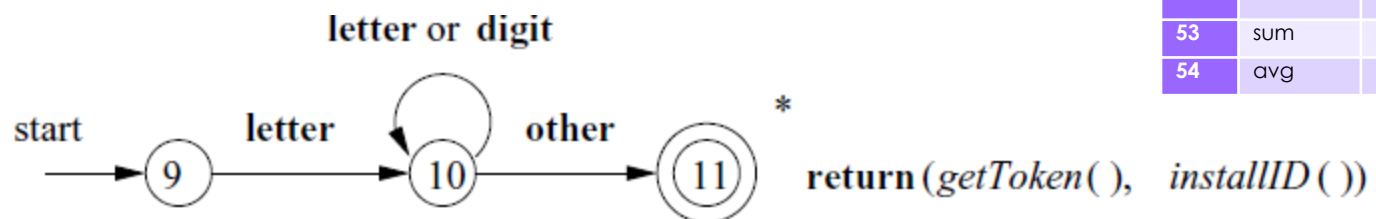
Recognition of Reserved Words and Identifiers

➤ There are two ways that we can handle reserved words that look like identifiers:

1. Install the reserved words in the symbol table initially Cont...

The function *getToken* examines the symbol table entry for the lexeme found, and returns whatever token name the symbol table says this lexeme represents — either **id** (or ID) or **one of the keyword tokens that was initially installed in the table.**

	lexeme name	token name
1	if	IF
2	then	THEN
3	else	ELSE
4	...	...
53	sum	ID
54	avg	ID



Recognition of Tokens

Recognition of Reserved Words and Identifiers

2. Create separate transition diagrams for each keyword.

- An example for the keyword then is shown in Fig. 3.15.
- A transition diagram of Fig. 3.15 consists of states representing the situation after each successive letter of the keyword is seen, followed by a test for a "nonletter-or-digit," i.e., any character that cannot be the continuation of an keyword/identifier.
- It is necessary to check that the identifier has ended, or else we would return token then (as keyword) in situations where the correct token was id, with a lexeme like **thennextvalue** that has **then** as a proper prefix.

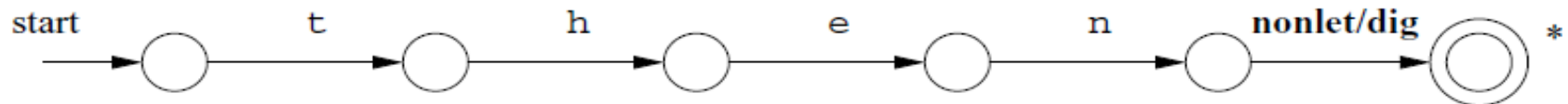


Figure 3 15 Hypothetical transition diagram for the keyword then

Recognition of Tokens

Recognition of Reserved Words and Identifiers

2. Create separate transition diagrams for each keyword. **Cont...**

If we adopt this approach, then **we must prioritize the tokens so that the reserved-word tokens are recognized in preference to id, when the lexeme matches both patterns. We do not use this approach in our example, which is why the states in Fig. 3.15 are unnumbered.**

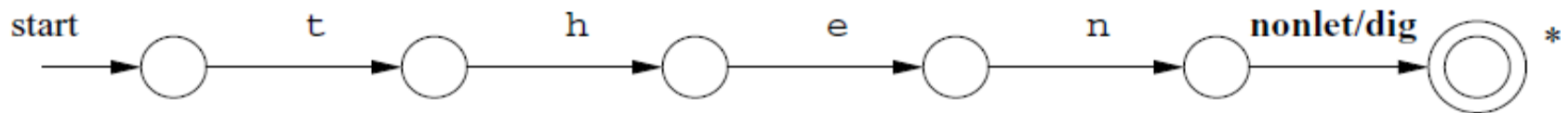


Figure 3 15 Hypothetical transition diagram for the keyword then

Recognition of Tokens

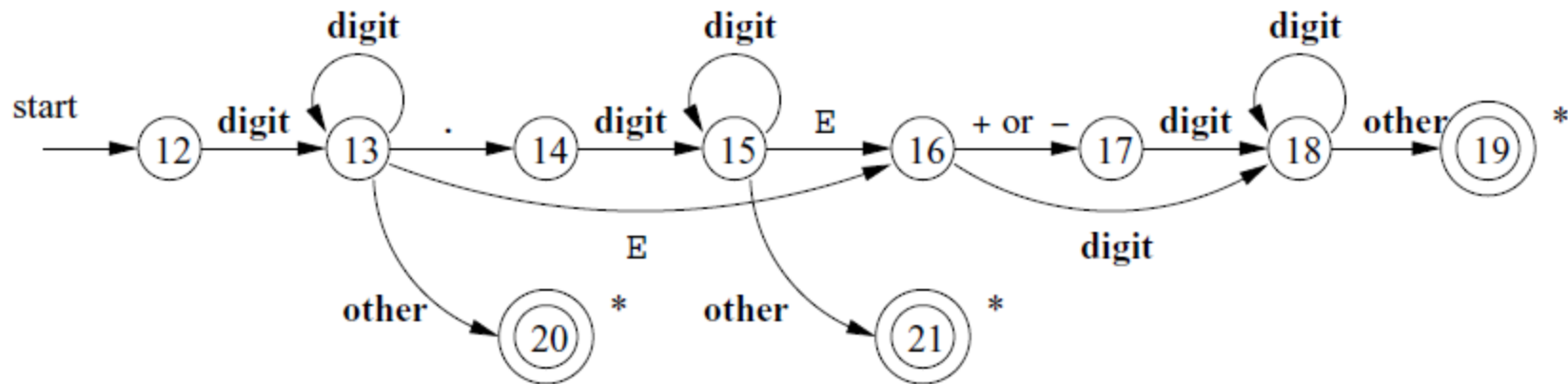
Transition Diagrams

- **Figure 3.16:** A transition diagram for unsigned numbers
- **Figure 3.17:** A transition diagram for whitespace

Recognition of Tokens

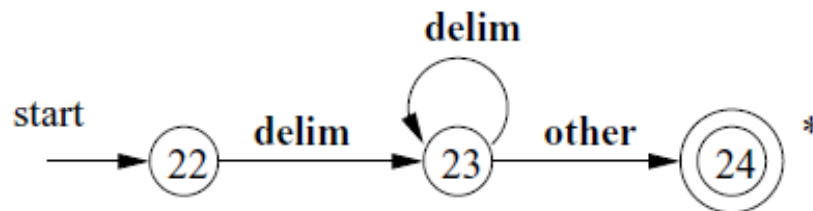
Transition Diagrams

- **Figure 3.16:** A transition diagram for unsigned numbers



Check for examples 15567, 55.99, 5.105E255, 6.22E-88, 25E500 ...

- **Figure 3.17:** A transition diagram for whitespace



`delim` $\rightarrow$ `[ \t\v\n\f ]`

`ws` $\rightarrow$ `{delim}+`

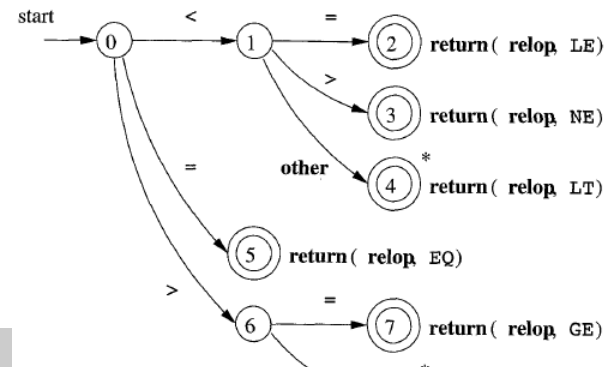
Recognition of Tokens

Architecture of a Transition-Diagram-Based Lexical Analyzer

- There are **several** ways that a collection of transition diagrams can be used to build a lexical analyzer.
- Each state is represented by a piece of code. (fig. 3.18)
- We may imagine a **variable state** holding the **number** of the current state for a transition diagram.
- A **switch based** on the value of **state** takes us to code for each of the possible states, where we find the action of that state.

Often, the code for a state is itself a switch statement or multiway branch that determines the next state by reading and examining the next input character.

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
               or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                   retToken.attribute = GT;
```



Recognition of Tokens

Architecture of a Transition-Diagram-Based Lexical Analyzer

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

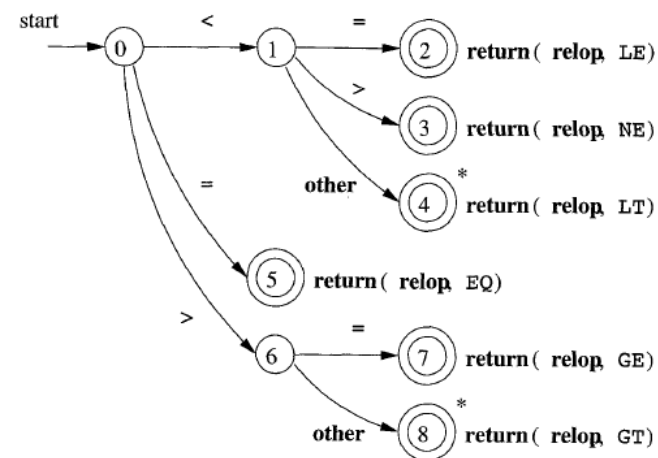


Figure 3.13: Transition diagram for `relop`

Figure 3.18: Sketch of implementation of `relop` transition diagram

Example 3.10 : In Fig. 3.18 we see a sketch of `getRelop()`, a C++ function whose job is to simulate the transition diagram of Fig. 3.13 and return an object type `TOKEN`, that is, a pair consisting of the token name (which must be `relop` in this case) and an attribute value (the code for one of the six comparison operators in this case).

The Lexical-Analyzer Generator *Lex/Flex*

<https://gnuwin32.sourceforge.net/packages/flex.htm>

<https://silcnitc.github.io/lex.html>

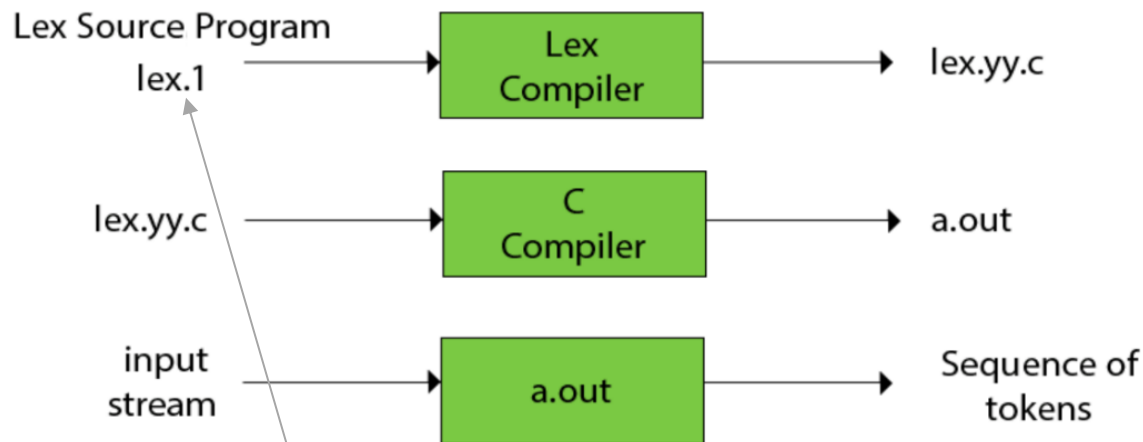
C:\GnuWin32\bin\Lex Pgms

The Lexical-Analyzer Generator Lex

- **Lex/Flex is a tool for generating lexical analyzers.**
- **It uses regular expressions to define patterns for tokens.**
- **It reads a specification file (with .l extension) and produces a C program that performs lexical analysis.**
- **The generated lexical analyzer is typically invoked through the function yylex(), which scans input and returns tokens to the parser.**
- **Note:**
 - **Flex (Fast LEX) is an improved, faster, and more modern version of LEX.**
 - **Flex is widely used today because it is more efficient and supports additional features.**
 - **Flex is backward compatible with LEX, so most LEX programs work with Flex.**

The Lexical-Analyzer Generator Lex

Lex Specification execution steps



The input notation for the Lex tool is referred to as the Lex language and the tool itself is the Lex compiler.

Behind the scenes, the Lex compiler transforms the input patterns (i.e., Regex) into a transition diagram and generates code, in a file called `lex.yy.c`, that simulates this transition diagram.

The Lexical-Analyzer Generator Lex

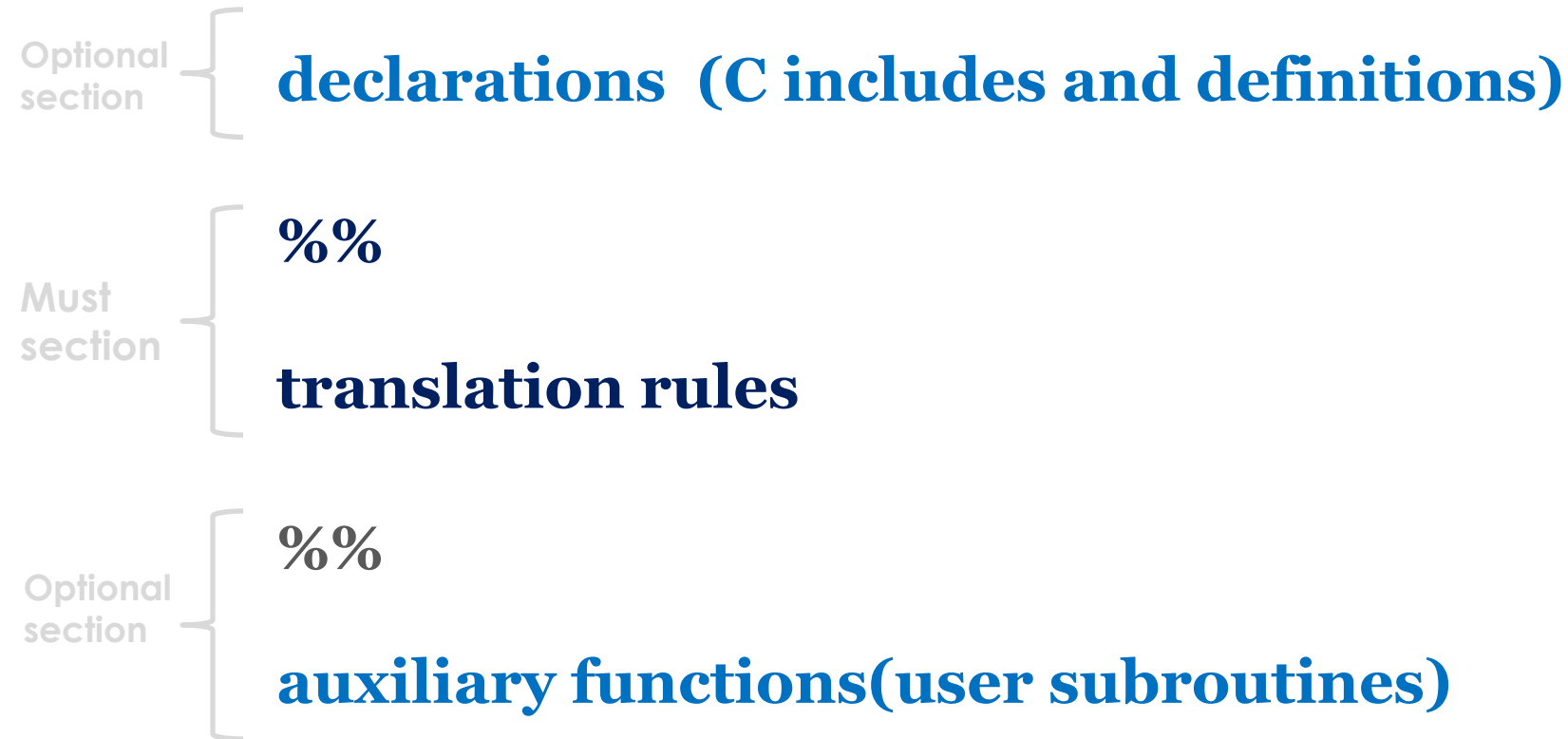
Lex/Flex Converts Regex $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ C code

Internally, Lex uses **automata theory**:

- 1. Regular Expressions $\rightarrow$ NFA** (Non-deterministic Finite Automaton)
 - Each regex is converted into an NFA using **Thompson's construction**.
 - NFAs allow multiple possible transitions for the same input.
- 2. NFA $\rightarrow$ DFA** (Deterministic Finite Automaton)
 - Lex applies the **subset construction algorithm** to convert NFA to DFA.
 - DFA has a single unique transition for each input symbol.
- 3. DFA $\rightarrow$ Optimized DFA**
 - Lex minimizes the DFA using **state minimization algorithms** for efficiency.
- 4. DFA Execution**
 - The generated C code (lex.yy.c) contains a **table-driven DFA**.
 - When yylex() runs:
 - It reads input character by character.
 - Traverses DFA states until no further transition is possible.
 - The longest match wins (Lex uses **maximal munch rule**).
 - Executes the corresponding action for the matched token.

The Lexical-Analyzer Generator Lex

- A Lex program has the following 3-sections:



The Lexical-Analyzer Generator Lex

➤ **The declarations section** includes

1. **declaration of variables** and **header files** (e.g., #include <stdio.h>)
2. **manifest constants** (i.e., #define constants),

The above mentioned will be written inside a pair of special delimiters, **%{** and **%}**. **Anything within these brackets is copied directly to the file lex.yy.c** and is not treated as a regular definition.

3. **regular definitions.**

➤ **Regular Definitions example**

```
/* regular definitions */
delim      [ \t\n]
ws         {delim}+
letter     [A-Za-z]
digit      [0-9]
id         {letter}({letter}|{digit})*
number     {digit}+(\.{digit}+)?(E[+-]?{digit}+)?
```

Example:

```
%{
#define YYSTYPE char*
#include "y.tab.h"
#include <stdio.h>
extern void yyerror(const char *);
extern YYSTYPE yylval;
%}
```

Note: Regular definitions that are used in later definitions or in the patterns of the translation rules are surrounded by curly braces.

➤ Translation rules : line oriented:

<pattern> <whitespace> <action>

- Each pattern is a regular expression for tokens, which may use the regular definitions of the declaration section.

<pattern> : starts at beginning of line, continues up to first unescaped whitespace.

- The actions are fragments of code, typically written in C. Action is performed when a pattern matches.

<action> : a single C statement. (if multiple statements, then they are enclosed in braces { }).

- **unmatched input characters** : copied to stdout.

The Lexical-Analyzer Generator Lex

- The **final section** is the **user subroutines section**,
 - **which consists of any legal C code.**
 - **Lex copies it to the C file after the end of the lex generated code.**

We have included a `main()` program.

```
%%  
main( )  
{  
    yylex();  
}
```

- The **lexer produced by Lex tool** is a C routine called **yylex()**, so we call it. Unless the actions contain explicit return statements, `yylex()` won't return until it has processed the entire input.

The Lexical-Analyzer Generator Lex

Anything between these
2 marks is copied as it is
in lex.yy.c

```
%{  
    /* definitions of manifest constants  
    LT, LE, EQ, NE, GT, GE,  
    IF, THEN, ELSE, ID, NUMBER, RELOP */  
%}  
  
/* regular definitions */  
delim    [ \t\n]  
ws       {delim}+  
letter   [A-Za-z]  
digit    [0-9]  
id       {letter}{letter|{digit}}*  
number   {digit}+(\.{digit})?(E[+-]?{digit})?
```

braces means the pattern
is defined somewhere

pattern

```
%%  
{ws}      { /* no action and no return */  
if         {return(IF);}  
then       {return(THEN);}  
else       {return(ELSE);}  
{id}      {yylval = (int) installID(); return(ID);}  
{number}  {yylval = (int) installNum(); return(NUMBER);}  
"<"       {yylval = LT; return(RELOP);}  
"<="      {yylval = LE; return(RELOP);}  
"="        {yylval = EQ; return(RELOP);}  
">="      {yylval = GE; return(RELOP);}  
">"       {yylval = GT; return(RELOP);}  
">="      {yylval = GE; return(RELOP);}  
%%
```

Actions

```
int installID() { /* function to install the lexeme, whose  
                  first character is pointed to by yytext,  
                  and whose length is yyleng, into the  
                  symbol table and return a pointer  
                  thereto */  
}  
  
int installNum() { /* similar to installID, but puts numer-  
                    ical constants into a separate table */  
}
```

- **The lexical analyzer created by Lex behaves in concert with the parser as follows.**
- **When called by the parser, the lexical analyzer begins reading its remaining input, one character at a time, until it finds the longest prefix of the input that matches one of the patterns P_i . It then executes the associated action A_i .**
- **Typically, A_i will return to the parser**, but if it does not (e.g., because P_i describes whitespace or comments), then the lexical analyzer proceeds to find additional lexemes, until one of the corresponding actions causes a return to the parser.
- **The lexical analyzer returns a single value, the token name, to the parser**, but uses the shared, integer variable **yylval** to pass additional information about the lexeme found, if needed.

The Lexical-Analyzer Generator Lex

Expression	Description
<code>\x</code>	x, if x is a lex operator. Example: <code>\.</code>
<code>"xy"</code>	xy, even if x or y is a lex operator (except <code>\</code>)
<code>[xy]</code>	x or y
<code>[x-z]</code>	x, or y, or z. Shorthand for <code>x y z</code>
<code>[^x]</code>	Any character but x
<code>.</code>	Any character but newline
<code>^x</code>	x at the beginning of a line
<code><y>x</code>	x when lex is in start condition/state y
<code>x\$</code>	x at the end of a line
<code>x?</code>	Optional x
<code>x*</code>	0, 1, 2, ... instances of x
<code>x+</code>	1, 2, 3, ... instances of x
<code>x{m,n}</code>	m through n occurrences of x

The Lexical-Analyzer Generator Lex

Expression	Description
$x\{n\}$	exactly n occurrences of x
$x\{n, \}$	n or more occurrences of x
$[...]$	One of the characters in the brackets. Example: $T[ao]p$ Sample match: Tap or Top
$[x-z]$	One of the characters in the range from x to z
$[^x-z]$	One of the characters not in the range from x to z
$xx yy$	Either xx or yy
$x \mid$	The action on x is the action for the next rule
(x)	x
x/y	x but only if followed by y
$\{defn\}$	substitute regular definition

The Lexical-Analyzer Generator Lex

Operators	Description
?	Zero or one copy of the preceding expression
*	Zero or more copies of the preceding expression
+	One or more copies of the preceding expression

Expression	Description
a b	a or b (alternating)
(ab)+	One or more copies of ab (grouping)
abc	abc
abc*	ab abc abcc abccc abcccc ...
abc+	abc abcc abccc abcccc ...
a(bc)+	abc abcbc abcbcbc ...
a(bc)*	a abc abcbc abcbcbc ...
a(bc)?	a abc (none or once)

The Lexical-Analyzer Generator Lex

Expression	Description
[x-z]	- Matches one of the characters in the range from x to z - Matches x, or y, or z. Shorthand for x y z
[xyz]	Matches x, or y, or z
[x\ -z]	Matches x, or -, or z
[A-Za-z0-9]+	Matches one or more alphanumeric characters
[^x]	Matches any character except x
[^x-z]	- Matches one of the characters not in the range from x to z (Anything except x or y or z) - The ^ prefix in a character class creates a complement class.
[x^z]	One of x, ^, z
[x z]	One of x, , z
.	Matches any (single) character except newline

The Lexical-Analyzer Generator Lex

LEX common actions

;	→	Null action (i.e., do nothing).
ECHO;	→	printf("%s", yytext);
{...}	→	Multi-statement action.
return yytext;	→	send contents of yytext to the parser.

➤ **yytext**: C-String of matched characters (Make a copy if necessary!)

➤ **yytext**: Length of the matched characters.

➤ **ECHO**

- a macro that writes characters matched by the pattern.
- This is the **default action for any unmatched strings**.
- Typically, ECHO is defined as:

#define ECHO fwrite(yytext, yytext, 1, yyout)

The Lexical-Analyzer Generator Lex

Example:

Figure 3.23 is a Lex program that recognizes the tokens of Fig. 3.12 and returns the token found.

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any ws	-	-
if	if	-
then	then	-
else	else	-
Any id	id	Pointer to table entry
Any number	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
>	relop	NE
>=	relop	GT

Figure 3.12: Tokens, their patterns, and attribute values

declarations

```
%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
}%

/* regular definitions */
delim    [ \t\n]
ws       {delim}+
letter   [A-Za-z]
digit    [0-9]
id       {letter}({letter}|{digit})*
number   {digit}+(\.{digit}+)?(E[+-]?{digit}+)?
```

translation rules

```
%%

{ws}      { /* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yyval = (int) installID(); return(ID);}
{number}  {yyval = (int) installNum(); return(NUMBER);}
"<"      {yyval = LT; return(RELOP);}
"<="     {yyval = LE; return(RELOP);}
"="       {yyval = EQ; return(RELOP);}
">"      {yyval = NE; return(RELOP);}
">="     {yyval = GT; return(RELOP);}
">="     {yyval = GE; return(RELOP);}
```

auxiliary functions

```
%%

int installID() { /* function to install the lexeme, whose
                  first character is pointed to by yytext,
                  and whose length is yyleng, into the
                  symbol table and return a pointer
                  thereto */
}

int installNum() { /* similar to installID, but puts numer-
                   ical constants into a separate table */
}
```

Figure 3.23: Lex program for the tokens of Fig. 3.12

The Lexical-Analyzer Generator Lex

Anything between these
2 marks is copied as it is
in lex.yy.c

```
%{  
    /* definitions of manifest constants  
    LT, LE, EQ, NE, GT, GE,  
    IF, THEN, ELSE, ID, NUMBER, RELOP */  
%}  
  
/* regular definitions */  
delim    [ \t\n]  
ws       {delim}+  
letter   [A-Za-z]  
digit    [0-9]  
id       {letter}{letter|{digit}}*  
number   {digit}+(\.{digit})?(E[+-]?{digit})?
```

braces means the pattern
is defined somewhere

pattern

```
%  
{ws}      { /* no action and no return */  
if         {return(IF);}  
then       {return(THEN);}  
else       {return(ELSE);}  
{id}      {yylval = (int) installID(); return(ID);}  
{number}  {yylval = (int) installNum(); return(NUMBER);}  
"<"       {yylval = LT; return(RELOP);}  
"<="      {yylval = LE; return(RELOP);}  
"="        {yylval = EQ; return(RELOP);}  
">="      {yylval = GE; return(RELOP);}  
">"       {yylval = GT; return(RELOP);}  
">="      {yylval = GE; return(RELOP);}  
%  
%%
```

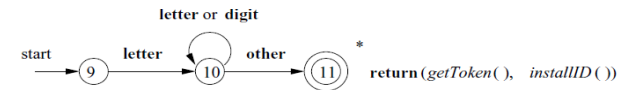
Actions

```
int installID() { /* function to install the lexeme, whose  
                  first character is pointed to by yytext,  
                  and whose length is yyleng, into the  
                  symbol table and return a pointer  
                  thereto */  
}  
  
int installNum() { /* similar to installID, but puts numer-  
                    ical constants into a separate table */  
}
```

The Lexical-Analyzer Generator Lex

■ The action taken when id is matched is threefold:

1. Function **installID()** is called to place the lexeme found into the symbol table.



2. **installID()** function returns a pointer to the symbol table, which is placed in global variable **yylval**, where it can be used by the parser or a later component of the compiler.

3. Note that **installID()** has **two variables** that are set automatically by the lexical analyzer that Lex tool generates:

- a) **yytext** is a pointer to the beginning of the lexeme, analogous to lexemeBegin.
- b) **yyleng** is the length of the lexeme found.

4. The token name **ID** is returned to the parser.

Note: **yylval** is a global variable that holds a pointer to the symbol table.

The Lexical-Analyzer Generator Lex

- **The action taken when a lexeme matching the pattern number** is similar to identifier, using the auxiliary function **installNum()**.
- **retract()** - If the accepting state has a *, this function is used to retract forward pointer.
- **getToken()** - examines the symbol table entry for the lexeme found and returns the corresponding token name.
- **fail()** - resets forward ptr to lexeme_begin to try another transition diagram.
 - What the fail() function does depends on the global error recovery strategy of the Lexical Analyzer

Conflict Resolution in Lex (or Ambiguity elimination in LEX)

- **Choosing between different possible matches (Maximum Munch Rule):**

When more than one pattern (or regex) matches the input, lex chooses as follows:

- 1. The longest match is preferred.**
- 2. Among rules that match the same number of characters, the rule that occurs earliest in the list is preferred.**

The Lexical-Analyzer Generator Lex

■ Four special options for actions:

|, ECHO, BEGIN, and REJECT;

■ | indicates that the action for this rule is from the action for the next rule

[\t\n]	;	← Semicolon indicates null action
" "		
"\t"		
"\n"	;	

■ The unmatched token is using a default action that ECHO from the input to the output.

Start conditions

- ❑ Start conditions are **a mechanism for conditionally activating patterns.**
- ❑ This is useful for handling –
 - Conceptually different components of an input
 - Situations where the lex defaults (e.g., “longest possible match”) don’t work, like in comments or quoted strings.
- ❑ Declare a set of start condition names using

%start name1 name2 . . .

If **name1** is a start condition name, then a pattern prefixed with <name1> will only be active/match when the scanner is in start condition **name1**.

Example: removeCommentLines.l

The Lexical-Analyzer Generator Lex

BEGIN

- BEGIN is used to **switch Lex to a different start condition (state)** during scanning.
- Lex supports **start conditions** to handle context-sensitive scanning. For example, you might want different token rules when inside a comment or a string.
- You define start conditions using %s or %start in the **definitions section**.
- Example:

```
%s COMMENT
```

```
%%
```

```
"/*"          { BEGIN(COMMENT); }    // Enter COMMENT state  
<COMMENT>"/" { BEGIN(INITIAL); }    // Return to normal state  
<COMMENT>.    { /* ignore comment text */ }
```

REJECT (a special directive used inside the action block)

- REJECT tells Lex to **ignore the current match and continue searching for another possible match starting at the same input position.**
- Useful when multiple patterns can match the same input and you want **all applicable actions** to run.

Example:

%%

```
[0-9]+    { printf("Number: %s\n", yytext); REJECT; }  
[0-9]{2}  { printf("Two-digit number: %s\n", yytext);}
```

- If input is 123:
 - First rule matches 123 → prints "Number: 123".
 - REJECT forces Lex to try other rules at the same position.
 - Second rule matches 12 → prints "Two-digit number: 12".

Lex Predefined Variables

- ❑ **yytext** -- a string containing the lexeme
- ❑ **yytext** -- the length of the lexeme
- ❑ **yyin** -- the input stream pointer
 - the default input of default main() is **stdin**
- ❑ **yyout** -- the output stream pointer
 - the default output of default main() is **stdout**.
- ❑ **%./a.out < inputfile > outfile**
- ❑ E.g.
 - ❑ `[a-z]+` `printf("%s", yytext);`
 - ❑ `[a-z]+` `ECHO;`
 - ❑ `[a-zA-Z]+` `{words++; chars += yytext;}`

□ `yylex()`

- The LEX tool automatically defines/produce a C routine called `yylex()` in `lex.yy.c`, but does not call it.
- The programmer must call the lexer `yylex()` in the Auxiliary functions section of the LEX program.
- LEX generates code for the definition of `yylex()` according to the rules specified in the Rules section.
- Unless the actions contain explicit return statements, `yylex()` won't return until it has processed the entire input.

Lex Library Routines

□ **yymore()**

- Appends the next matched string to the current value of the yytext array rather than replacing the contents of the yytext array.

□ **yyless(n)**

- Retains **n initial characters in the yytext array** and returns the remaining characters to the input stream.

□ **yywarp()**

- yywrap() is called by lex, when input is exhausted.
- The default yywarp() always returns 1.

Lex Library Routines

Note:

- Lex was designed to work with Yacc (Yet Another Compiler Compiler), which handles **syntax analysis** while Lex handles **lexical analysis**.
- **The naming convention using yy** (like yylex, yyparse, and lex.yy.c) was chosen simply to create a **namespace** for functions and variables used by Lex and Yacc, avoiding conflicts with user-defined names.
- The **lex.yy.c** file name indicates that the output of Lex is intended to integrate with a Yacc-generated parser.

The Lexical-Analyzer Generator Lex



- Write a lex specification to echo all strings of capital letters, followed by a space tab (`\t`) or newline (`\n`) dot (`\.`) or comma (`\,`) to stdout, and all other characters will be ignored.

The Lexical-Analyzer Generator Lex

- Write a lex specification to echo all strings of capital letters, followed by a space, tab (\t), newline (\n), dot (\.) or comma (\,) to stdout, and all other characters will be ignored.

```
/* echo-upcase-words.l */
```

```
%%
```

```
[A-Z]+[ \t\n\.\,]
```

```
printf("%s",yytext);
```

```
.
```

```
; /* no action specified */
```

The Lexical-Analyzer Generator Lex

➤ Example:

Figure 3.23 is a Lex program that recognizes the tokens of Fig. 3.12 and returns the token found.

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any ws	-	-
if	if	-
then	then	-
else	else	-
Any id	id	Pointer to table entry
Any number	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
>	relop	NE
>=	relop	GT

Figure 3.12: Tokens, their patterns, and attribute values

```

%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
}%

/* regular definitions */
delim    [ \t\n]
ws       {delim}+
letter   [A-Za-z]
digit    [0-9]
id       {letter}({letter}|{digit})*
number   {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%

{ws}     { /* no action and no return */}
if       {return(IF);}
then     {return(THEN);}
else     {return(ELSE);}
{id}     {yylval = (int) installID(); return(ID);}
{number} {yylval = (int) installNum(); return(NUMBER);}
"<"     {yylval = LT; return(RELOP);}
"<="    {yylval = LE; return(RELOP);}
"="      {yylval = EQ; return(RELOP);}
">"     {yylval = NE; return(RELOP);}
">="    {yylval = GT; return(RELOP);}
">"     {yylval = GE; return(RELOP);}

%%

int installID() { /* function to install the lexeme, whose
                  first character is pointed to by yytext,
                  and whose length is yyleng, into the
                  symbol table and return a pointer
                  thereto */
}

int installNum() { /* similar to installID, but puts numer-
                   ical constants into a separate table */
}
    
```

declarations

translation rules

auxiliary functions

Figure 3.23: Lex program for the tokens of Fig. 3.12

Conflict Resolution in Lex (or Ambiguity elimination in LEX)

Example 3.12:

- The first rule tells us to continue reading letters and digits to **find the longest prefix** of these characters to group as an identifier. It also tells us to treat `<=` as a single lexeme, rather than selecting `<` as one lexeme and `=` as the next lexeme.
- The second rule makes keywords reserved, if **we list the keywords before id in the program**. For instance, if *then* is determined to be the longest prefix of the input that matches any pattern, and the pattern *then* precedes `{ id }`, as it does in Fig. 3.23, then the token THEN is returned, rather than ID.

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

a*b printf("1");

ca printf("2");

a*ca* printf("3");

➤ Given the following input string

abcaa cacaaabbbaabcaaca

➤ What does the lexer print? Clearly indicate all the tokens.

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

a*b printf("1");

ca printf("2");

a*ca* printf("3");

- Given the following input string

abcaa cacaaabbbaabcaaca

- What does the lexer print? Clearly indicate all the tokens.

Output: 13 2311132

Tokens : ab caa ca caaa b b aaab caa ca

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

aa printf("1");

a printf("2");

ab printf("3");

- Given the following input string: aab
- What does the lexer print? Clearly indicate all the tokens.

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

aa **printf(“1”);**

a **printf(“2”);**

ab **printf(“3”);**

- Given the following input string: aab
- What does the lexer print? Clearly indicate all the tokens.

Answer: 1b

Tokens: aa

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

`%%`

`a*b` `printf(" 1 ");`

`(a|b)*b` `printf(" 2 ");`

`c*` `printf(" 3 ");`

- Give an example of an input to this scanner that will produce **132** as an output, or explain why one does not exist.

- Explain how lex would tokenize the following input string and the output produced.
 - (i) cbbbbac (ii) cbabc

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

a*b **printf(“ 1 ”);**

(a|b)*b **printf(“ 2 ”);**

c* **printf(“ 3 ”);**

- Give an example of an input to this scanner that will produce **132** as an output, or explain why one does not exist.

Solution: abcbb, abccbb, aabccbb, ...

- Explain how lex would tokenize the following input string and the output produced.

(i) cbbbbac

(ii) cbabc

Solution:

(i) 32a3 (c - 3 bbbb - 2 a - a c - 3)

(ii) 323 (c - 3 bab - 2 c - 3)

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

(01|10) printf(“louder”);

o(01)*1 printf(“Actions ”);

(1010*1|0101*o) printf(“speak ”);

- Give an input to this scanner such that the output string is:

(Actions⁴ speak²)³ louder³

- Where, Aⁱ denotes A repeated i times. (And, of course, the parentheses are not part of the output.) You may use similar shorthand notation in your answer.

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

(01|10) printf(“louder”);

o(01)*1 printf(“Actions ”);

(1010*1|0101*o) printf(“speak ”);

- Give an input to this scanner such that the output string is:

(Actions⁴ speak²)³ louder³

- Where, Aⁱ denotes A repeated i times. (And, of course, the parentheses are not part of the output.) You may use similar shorthand notation in your answer.

Solution: ((0011)⁴(0100)²)³011001

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

aa **printf("1")**

b?a+b? **printf("2")**

b?a*b? **printf("3")**

➤ **Questions:**

- Provide output for the String : **bbbaabb**
- Provide a String such that the Output is **123**
- Provide a String such that the Output is **321**

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

aa printf("1")

b?a+b? printf("2")

b?a*b? printf("3")

➤ Questions:

➤ Provide output for the String : **bbbaabb**

➤ **323** (bb - 3 baab - 2 b - 3)

➤ Provide a String such that the Output is **123**

➤ **Not possible**

➤ Provide a String such that the Output is **321**

➤ **bbbabaa** (bb - 3 bab - 2 aa - 1 , Other answers are also possible)

The Lexical-Analyzer Generator Lex

Consider the following lexical specification:

%%

a?aab

printf("2")

a?b

printf("1")

aab

printf("3")

➤ **Questions:**

- Provide output for the String : **aab**
- Provide a String such that the Output is **321**
- Which of the aforementioned rule is useless and why?

The Lookahead Operator

- Lex reads one character ahead of the last character that forms the selected lexeme, and then retracts the input so only the lexeme itself is consumed from the input.
- **Sometimes, we need a certain pattern to be matched to the input only when it is followed by a certain other characters.**
(e.g. `ab/cd`)
 - If so, we may use the slash (`/`) in a pattern to indicate the end of the part of the pattern that matches the lexeme.
 - **What follows `/` is additional pattern that must be matched before we can decide that the token in question was seen, but what matches this second pattern is not part of the lexeme.**

The Lookahead Operator (/)

- ▣ The regular expression

ab/cd

matches the string ab, but only if followed by cd, but what matches this second pattern (cd) is not part of the lexeme.

The Lookahead Operator

- **Example 3.13 :** In Fortran and some other languages, **keywords are not reserved**. That situation creates problems, such as a statement

IF(I,J) = 3

where **IF is the name of an array, not a keyword**. This statement contrasts with statements of the form

IF(condition) THEN ...

where IF is a keyword. we can be sure that the **keyword IF is always followed by a left parenthesis, some text — the condition — that may contain parentheses, a right parenthesis and a letter**.

Thus, we could write a Lex rule for the keyword IF like:

This rule says that the pattern the lexeme matches is just the two letters IF. **The slash says that additional pattern follows but does not match the lexeme.**

The Lookahead Operator

- **Example 3.13 :** In Fortran and some other languages, **keywords are not reserved**. That situation creates problems, such as a statement

IF(I,J) = 3

where **IF is the name of an array, not a keyword**. This statement contrasts with statements of the form

IF(condition) THEN ...

where IF is a keyword. we can be sure that the **keyword IF is always followed by a left parenthesis, some text — the condition — that may contain parentheses, a right parenthesis and a letter**.

Thus, we could write a Lex rule for the keyword IF like:

IF /\(.*\) {letter}

This rule says that the pattern the lexeme matches is just the two letters IF. **The slash says that additional pattern follows but does not match the lexeme.**

The Lookahead Operator

- **Example 3.13 :** In Fortran and some other languages, **keywords are not reserved**. That situation creates problems, such as a statement

IF(I,J) = 3

where **IF is the name of an array, not a keyword**. This statement contrasts with statements of the form

IF(condition) THEN ...

where IF is a keyword. we can be sure that the **keyword IF is always followed by a left parenthesis, some text — the condition — that may contain parentheses, a right parenthesis and a letter**.

Thus, we could write a Lex rule for the keyword IF like:

IF /\(.*\) {letter}

This rule says that the pattern the lexeme matches is just the two letters IF. **The slash says that additional pattern follows but does not match the lexeme.**

Lexical Analysis

Design of a Lexical-Analyzer Generator

➤ The Structure of the Generated Analyzer

- **Figure 3.49** Overviews the architecture of a lexical analyzer generated by Lex.
- The program that serves as the lexical analyzer includes a fixed program that simulates an automaton.
- The rest of the lexical analyzer consists of components that are created from the Lex program by Lex itself.

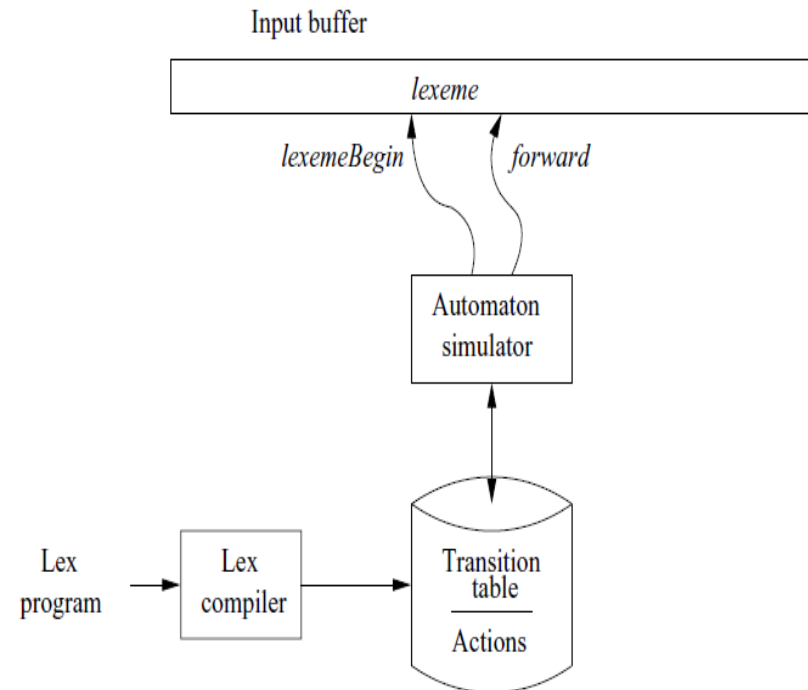


Figure 3.49: A Lex program is turned into a transition table and actions, which are used by a finite-automaton simulator

Design of a Lexical-Analyzer Generator

- **To construct the automaton,**
we begin by taking each regular-expression pattern in the Lex program and converting it, using Algorithm to convert a RE to an NFA.
- **We need a single automaton that will recognize lexemes matching any of the patterns in the program,**
so we combine all the NFA's into one by introducing a new start state with ϵ -transitions to each of the start states of the NFA's N_i for pattern p_i

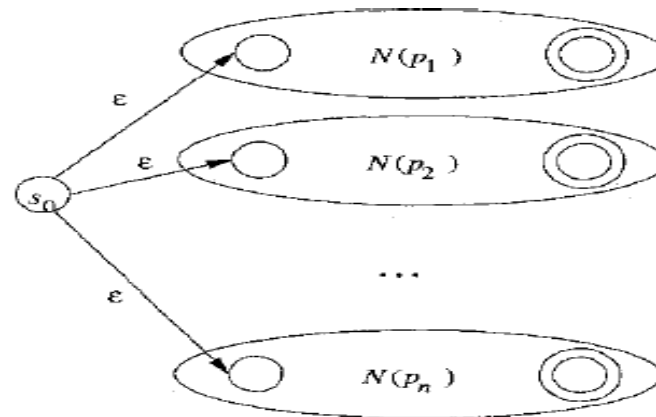


Figure 3.50: An NFA constructed from a **Lex** program

Design of a Lexical-Analyzer Generator

- **Example 3.26:** We shall illustrate the ideas of this section with the following simple, abstract example:

a { action *A1* for pattern *p1* }

abb { action *A2* for pattern *p2* }

a*b+ { action *A3* for pattern *p3* }

- Input string: **abb**
- Input string: **aabbb...**

Design of a Lexical-Analyzer Generator

a { action *A1* for pattern *p1* }

abb { action *A2* for pattern *p2* }

a*b+ { action *A3* for pattern *p3* }

- In particular, string **abb** matches both the second and third patterns, but we shall consider it a lexeme for pattern *p2*, since that pattern is listed first in the above Lex program.
- Input strings such as **aabbb...** have many prefixes (i.e., aab, aabb) that match the third pattern. The Lex rule is to take the longest, so we continue reading b's, until another *a* is met, whereupon we report the lexeme to be the initial a's followed by as many b's as there are.

Design of a Lexical-Analyzer Generator

a	{ action A1 for pattern p1 }
abb	{ action A2 for pattern p2 }
a*b+	{ action A3 for pattern p3 }



- Convert each regular expression to NFA

Figure 3.51: NFA's for a, abb, and a*b⁺



- Combine all NFAs as

Figure 3.52: Combined NFA

Design of a Lexical-Analyzer Generator

- Convert each regular expression to NFA

a
abb
a*b+

{ action A1 for pattern p1 }
{ action A2 for pattern p2 }
{ action A3 for pattern p3 }

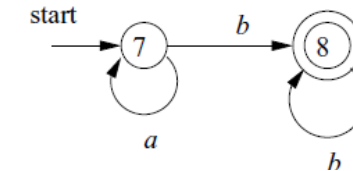
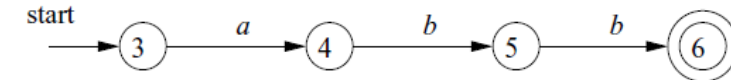
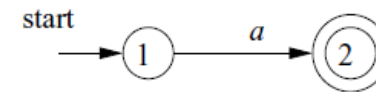


Figure 3.51: NFA's for a, abb, and a*b⁺

- Combine all NFAs as

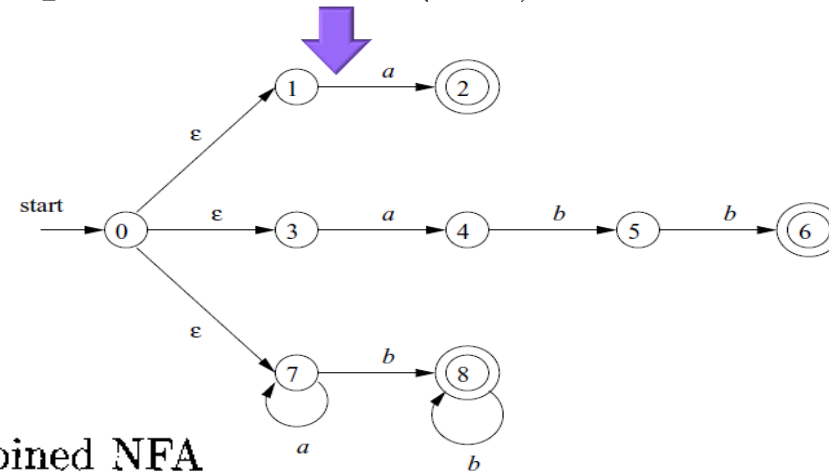


Figure 3.52: Combined NFA

Design of a Lexical-Analyzer Generator

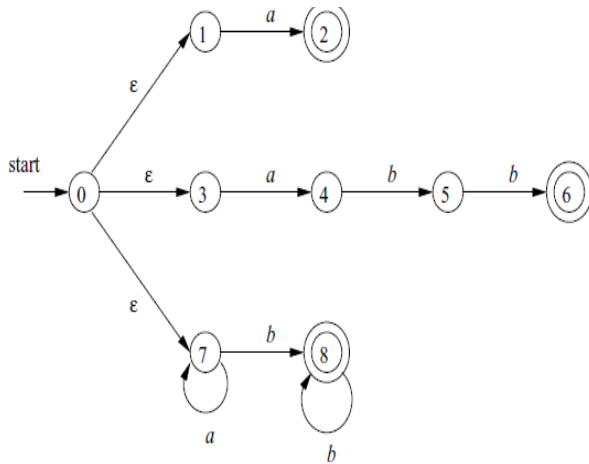


Figure 3.52 Combined NFA

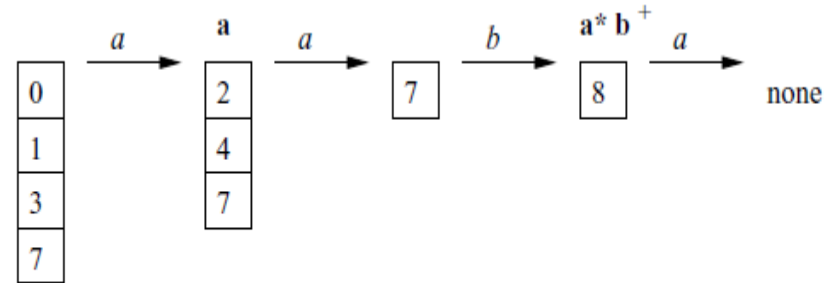


Figure 3.53 Sequence of sets of states entered when processing input *aaba*

❑ **Example 3.27:** Suppose the input begins **aaba**.

- Figure 3.53 shows the sets of states of the NFA of Fig. 3.52 that we enter, starting with ϵ -closure of the initial state 0, which is $\{0,1,3,7\}$, and proceeding from there.
- After reading the fourth input symbol, we are in an empty set of states, since in Fig. 3.52, there are no transitions out of state 8 on input *a*.
- Thus, we need to back up, looking for a set of states that includes an accepting state.

Design of a Lexical-Analyzer Generator

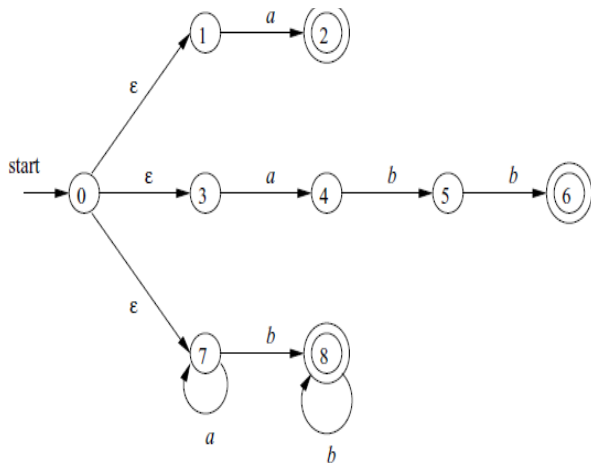


Figure 3.52 Combined NFA

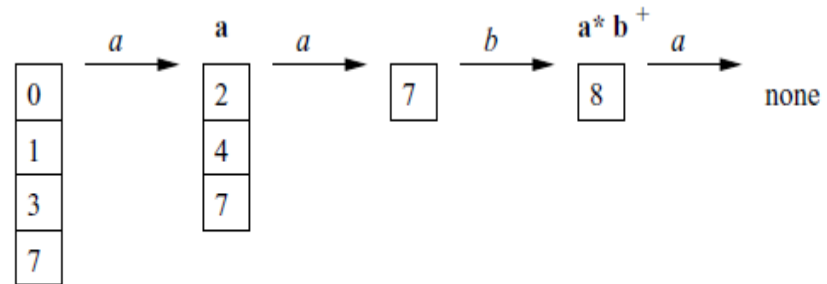


Figure 3.53 Sequence of sets of states entered when processing input *aaba*

- Notice that, as indicated in Fig. 3.53, after reading **a** we are in a set that includes state 2 and therefore indicates that the pattern **a** has been matched.
- However, after reading **aab**, we are in state 8, which indicates that **a*b⁺** has been matched; **prefix aab is the longest prefix that gets us to an accepting state.**
- **We therefore select aab as the lexeme, and execute action A₃, which should include a return to the parser indicating that the token whose pattern is $p_3 = a^*b^+$ has been found.**

Design of a Lexical-Analyzer Generator

DFA's for Lexical Analyzers

- Figure 3.54 shows a transition diagram based on the DFA that is constructed by the subset construction from the NFA in Fig. 3.52.
- The accepting states are labelled by the pattern that is identified by that state. For instance, the state $\{6,8\}$ has two accepting states, corresponding to patterns **abb** and **a^*b^+** . Since the former is listed first, that is the pattern associated with state $\{6,8\}$.

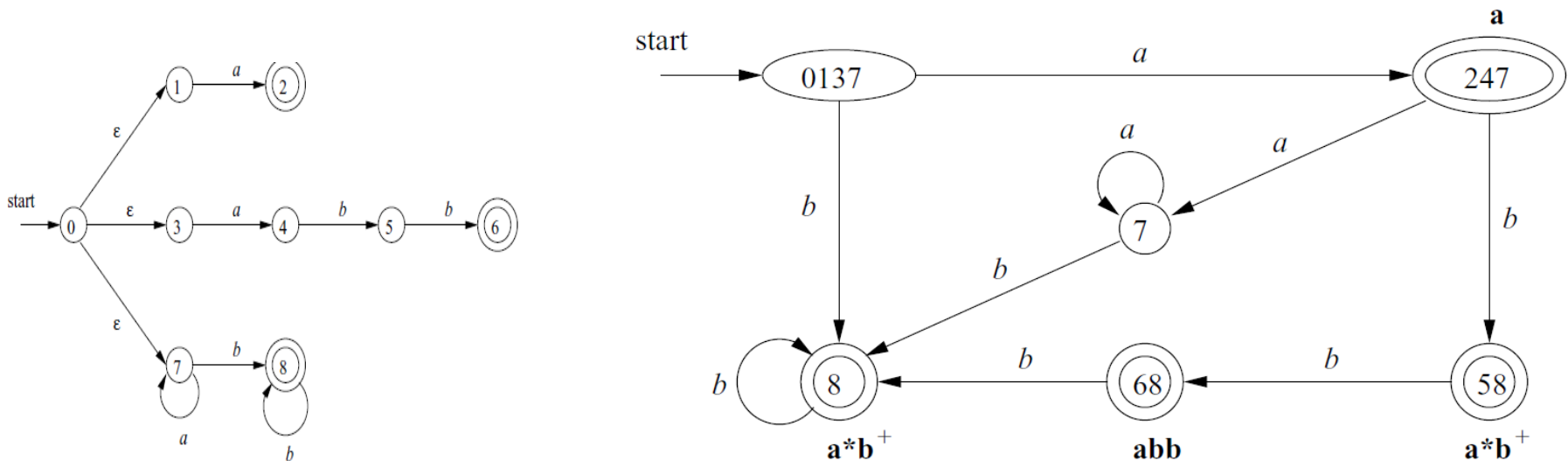


Figure 3.52 Combined NFA

Figure 3.54: Transition graph for DFA handling the patterns **a**, **abb**, and **a^*b^+**

Design of a Lexical-Analyzer Generator

- Using the following patterns explain the design of a lexical analyzer. Show the working of your lexical analyzer that simulates an NFA on the input string: **abaab**

ab {// some action}

aab {// some action}

aba {// some action}

Implementing the Lookahead Operator

- When converting the pattern r_1/r_2 to an NFA, **we treat the / as if it were ϵ , so we do not actually look for a / on the input.**

However, if the NFA recognizes a prefix **xy** of the input buffer as matching this regular expression, the end of the lexeme is not where the NFA entered its accepting state. Rather the end occurs when the NFA enters a state **s** such that

1. **s** has an ϵ -transition on the (imaginary) /,
2. There is a path from the start state of the NFA to state **s** that spells out **x**
3. There is a path from state **s** to the accepting state that spells out **y**.
4. **x** is as long as possible for any **xy** satisfying conditions 1-3.

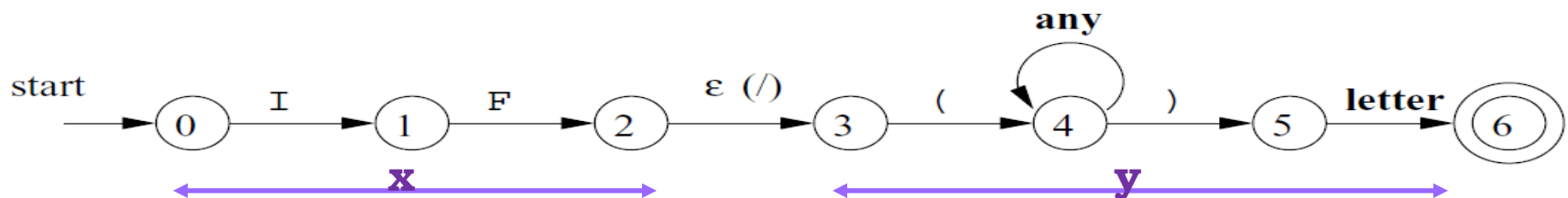


Figure 3.55: NFA recognizing the keyword IF

Design of a Lexical-Analyzer Generator

Implementing the Lookahead Operator

- An NFA for the pattern for the Fortran IF (branching statement) with lookahead is shown in Fig. 3.55.
- Notice that **the ϵ -transition from state 2 to state 3 represents the lookahead operator.**
- **State 6 indicates the presence of the keyword IF.**
- However, **we find the lexeme IF by scanning backwards to the last occurrence of state 2, whenever state 6 is entered.**

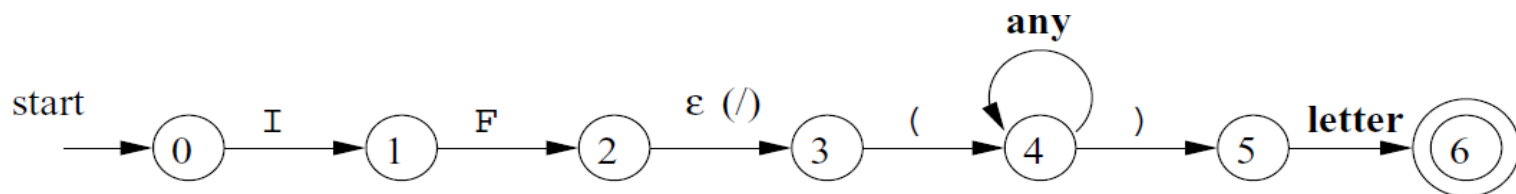


Figure 3.55: NFA recognizing the keyword IF

Lexical-Analyzer - Regex

Exercise: Write a Lex program to

1. Count the number of characters, words, and newlines.
2. Remove comment lines.
3. Identify identifiers (or generate tokens for identifiers)
4. Generate tokens for keywords in C.
5. Print line numbers (at the beginning of a line) for given file source file
6. Demonstrate lookahead operator.

Lexical-Analyzer - Regex

- Macros, Quantifiers, characters, Character Classes, Anchors and Boundaries.
- Mastering Quantifiers
 - **Greedy:** As Many As Possible (longest match)
 - **Docile:** Give Back When Needed
 - **Lazy:** As Few As Possible (shortest match)
 - **Helpful:** Expand When Needed
 - **Possessive:** Don't Give Up Characters
- The *Longest Match* and *Shortest Match* Traps
 - What does "longest match" mean?
 - What does "shortest match" mean?

References

- **Compilers—Principles, Techniques and Tools**, Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffery D. Ullman, 2nd Edition
- https://docs.python.org/3/reference/lexical_analysis.html
- <https://www.epaperpress.com/lexandyacc/prl.html>



THANK YOU

Prakash C O

Associate Professor,

Department of Computer Science & Engineering.

coprakasha@pes.edu

The Lexical-Analyzer Generator Lex

Name	Function
<code>int yylex(void)</code>	call to invoke lexer, returns token
<code>char *yytext</code>	pointer to matched string
<code>yylen</code>	length of matched string
<code>yyval</code>	value associated with token
<code>int yywrap(void)</code>	wrapup, return 1 if done, 0 if not done
<code>FILE *yyout</code>	output file
<code>FILE *yyin</code>	input file
<code>INITIAL</code>	initial start condition
<code>BEGIN</code>	condition switch start condition
<code>ECHO</code>	write matched string

Table 3: Lex Predefined Variables

Conflict Resolution in Lex (Maximum Munch Rule)

- We have alluded to the two rules that Lex uses to decide on the proper lexeme to select, **when several prefixes of the input match one or more patterns:**
 1. Always prefer a longer prefix to a shorter prefix.
 2. If the longest possible prefix matches two or more patterns, prefer the pattern listed first in the Lex program.

Tricky Problems When Recognizing Tokens

- In some languages it is not immediately apparent when we have seen an instance of a lexeme corresponding to a token.
- The following example is taken from Fortran. In the below statement

```
DO 5 I = 1.25
```

```
do label var = expr1, expr2, expr3  
statements  
label continue
```

it is not apparent that the first lexeme is **DO5I**, an instance of the identifier token, until we see the dot following the digit 1.

(This lexical analysis process needs many characters lookahead in determining a right lexeme).

Note that **blanks in fixed-format Fortran are ignored.**

Tricky Problems When Recognizing Tokens

- Had we seen a comma instead of the dot, we would have had a do-statement

DO 5 I = 1,25

```
do label var = expr1, expr2, expr3
statements
label continue
```

in which the first lexeme is the keyword DO.

Note: Standard Fixed Format

The standard fixed format source lines are defined as follows:

- A fixed source form line is a sequence of 1 to 132 characters. The default line size is 72 characters.
- Columns 1 through 5 contain either blanks, a statement label
- Column 6 contains a blank or zero.
- Columns 7 through to the right margin contain statement text, possibly followed by other statements or by an inline comment.

The Lexical-Analyzer Generator Lex

➤ operators:

- ❑ `*` : **match zero or more times**, eg: `ab*c` $\rightarrow$ `ac`, `abc`, `abbc`, ...
- ❑ `+` : **match one or more times**, eg: `ab+c` $\rightarrow$ `abc`, `abbc`, ...
- ❑ `?` : **zero or one occurrence**, eg: `ab?c` $\rightarrow$ `abc`, `ac`
- ❑ `( )` : **grouping**, eg: `(ab)+` $\rightarrow$ `ab`, `abab`...
- ❑ `|` : **alternatives**, eg `ab|cd` $\rightarrow$ `ab`, `cd`
- ❑ `{n,m}` : **repetition**, eg `a{1,3}` $\rightarrow$ `a`, `aa`, `aaa`
- ❑ `{defn}` : **substitute defn** (from first section).

➤ operators (cont):

□ **The square brackets []** enclose a sequence of characters, termed a character class.

1) The square brackets [] matches any single character in the sequence.

2) The hyphen(-) in a character class denotes an inclusive range.
e.g: [0-9] matches any digit.

1) The circumflex (^) symbol at the beginning denotes negation.
e.g: [^0-9] matches any character that is not a digit.